Fast Simulation-Based Bayesian Estimation of Dynamic Models using Normalizing Flow Neural Networks

Cameron Fen*

February 21, 2022

This paper proposes a simulation-based deep learning Bayesian estimation procedure for the estimation of macroeconomic models that is able to derive posteriors and likelihoods even when the likelihood function is not tractable in a manner much faster than Metropolis Hastings. I demonstrate this procedure Bayesian estimating models solved with projection, perturbation, and value function iteration. This improvement works even with data that has a latent state space time structure, where unobserved variables affect sequential time steps. This allows for a general approach to Bayesian and MLE estimation of models solved via value function iteration and projection method. I demonstrate the validity of the approach by estimating a 10 parameter HANK model solved via the Reiter method that generates 861 covariates per time step, showing that this approach can handle high dimensional simulated data. Estimated on this HANK model, this approach is also 9x faster with better posteriors than Metropolis-Hastings. I also estimate the algorithm with an 11-parameter model solved via value function iteration, which cannot be estimated with Metropolis Hastings or even most conventional maximum likelihood estimators. This both addresses the computational expense of Metropolis-Hastings and allows projection and value function iteration techniques to be estimated in a Bayesian and MLE manner.

JEL Codes: C11, C68, C63, C45
Keywords: Neural Networks, Bayesian Inference, Dynamic Estimation, Simulation-Based Estimators

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Eric Jang and Florian Gunsilius for helpful feedback and Alisdair McKay, Toni Whited, Emil Lakkis, and Jeppe Druedahl for code. All errors are my own.

I. Introduction

In a field as quantitative as macroeconomics, Bayesian or maximum likelihood estimation (MLE) of dynamic models should be the norm. Bayesian methods allow the use of informative priors, perhaps derived by microdata to inform estimation and provide distributional information on parameters. MLE has efficiency advantages over alternatives like method of simulated moments and modeling advantages over maximum simulated likelihood. The reason these approaches are less used is for a two-fold reason. First, many solution methods don't yield complementary likelihood functions so Metropolis Hastings and maximum likelihood are off the table. Second, Metropolis Hastings estimation of Bayesian posteriors is computationally inefficient. My approach of using simulation based inference for Bayesian and MLE estimation significantly ameliorates both these issues.

If one cannot estimate via perturbation, Bayesian and MLE estimation often cannot be done and because of this options with less attractive qualities are used. In this case, solved models are usually either calibrated, estimated via method of simulated moments (MSM), or estimated in a simulated maximum likelihood fashion. Each approach has its own drawbacks. Calibration is not an algorithmic approach so it lacks both rigour and quantification tools like standard errors. MSM is not efficient unless the global identification criterion is met, which is both unverifiable and unlikely. Adding in measurement error for maximum simulated likelihood changes the model and makes it less realistic as practitioners often want to assume the error comes from economic shocks and not measurement. All the above approaches also only provide point estimates.

I propose a method from machine learning (Tejero-Cantero et al., 2020) that can allow Bayesian and full information MLE estimation of models without a likelihood function and has more than 10x speed advantages over a technique like Metropolis-Hastings Markov Chain Monte Carlo (MH-MCMC) in tests. Thus one can directly Bayesian estimated from the value function or projection solution in an efficient manner. Although the method is more general, this paper focuses on the case of dynamic macroeconomic structural models in a Bayesian setting, with an extension to the maximum likelihood setting. This generally extends Kaji, Manresa and Pouliot (2020) to the

Bayesian setting and when data is not iid. This approach works even when the data has latent state space time structure that induces correlations between time steps and this paper can be used to Bayesian and MLE estimate models that have kinks and discontinuities in the value or policy function which prevents estimation using perturbation techniques. Even in cases where perturbation can be used, because of speed and gains from parallelization over MH-MCMC, the approach still should be considered.

I next will discuss briefly the Sequential Neural Posterior Estimation (SNPE) algorithm (Greenberg, Nonnenmacher and Macke, 2019) that can perform likelihood free Bayesian and MLE estimation. SNPE uses a model relatively unknown in economics, the normalizing flow (Rezende and Mohamed, 2015), which is a powerful conditional density estimator. Flows can perform Bayesian estimation by learning the posterior conditional distribution $P(\theta|x)$ trained on samples from the joint, $x, \theta \sim P(x, \theta) = P(x|\theta)P(\theta)$. One can think of SNPE as an extension of the Kristensen and Shin (2012) approach to likelihood estimation, where they simulate data from the joint via the prior and the likelihood. Then they use a kernel density estimator to estimate the likelihood function. In my case, I use a normalizing flow to replace their kernel density estimator, which allows me to extend their result to full Bayesian inference and accommodate data with a latent state space structure.

The advantages of this model also include the ability to estimate a model without a likelihood function, a advantage this approach has over conventional MH-MCMC. By definition, the likelihood is not needed for likelihood free simulation based inference approaches. Like simulated method of moments (McFadden, 1989), one only needs to be able to sample from the model. This approach would allow techniques like value function iteration, projection and other models with intractable likelihoods to be estimated in a Bayesian and MLE fashion.

Second, computational cost is faster than MH-MCMC. MH-MCMC runs into diminishing returns when parellelized for macro modeling, with Herbst and Schorfheide (2015) pointing out that simulation trials show that using a 24 core machine speeds up computation by 8 times at best. More information on MH-MCMC Bayesian estimation is given in the appendix and section III.A.. In contrast, the simulation step in simulation-based inference can be trivially parallelized without any loss

in sample efficiency no matter how many CPUs run the task. Using python, the posterior of an RBC model can be derived in a couple of hours on a single 4 CPU i7 seventh generation computer. My 10 parameter HANK model discussed below, which solves in upwards of thirty seconds per iteration, takes 57 hours to fully estimate on a 12 CPU Ryzen 7 5900X. In comparison MH-MCMC requires 520 hours to produce inferior posterior samples. Like MCMC, one could need to simulate up to 1 million models in large 40+ parameter environments (Tejero-Cantero et al., 2020), but this can be parallelized with no loss in sample efficiency and is already faster before parallelization. The previously mentioned advantages of simulation-based inference has quickly made the technique one of the leading estimation methods in many fields of physics (Brehmer, 2021), (Cranmer, Brehmer and Louppe, 2020).

II. Literature Review

This literature review will cover two topics: On simulation-based methods, and the literature on solving dynamic models, particularly with kinks and discontinuities. I will discuss MH-MCMC and machine learning background in the following sections.

While simulation-based techniques in economics are efficient when the model is well specified, recent work has been done to improve robustness. One such avenue is efficient method of moment estimators using a spectrum of moments so, under some regularity conditions, the data is guaranteed to be in the support of distributions spanned by the moments[1] (Carrasco et al., 2007), (Altissimo and Mele, 2009). Additionally, like all the other approaches suggested, these methods have a difficult time dealing with models with latent time structure. Another approach is approximate Bayesian inference (Rubin, 1984) to estimate models solved via value function iteration techniques, to estimate these models. However, in a well specified model, this approach is only approximately Bayesian, unlike the set of algorithms I propose.

One could even consider a particle filter a simulation based technique for deriving a likelihood

1. There is also work in machine learning attempting to perform robust and efficient moment estimation by using a spectrum of moments, most well known is the Generative Moment Matching Network (Li, Swersky and Zemel, 2015)

function given an intractable integral (Fernández-Villaverde and Rubio-Ramírez, 2007), but this approach only works with dynamic models that yield complementary state space representations and requires a likelihood function at each point in time. Although the literature is more sparse, there are papers that also perform simulated Bayesian inference (Flury and Shephard, 2011) (Herbst and Schorfheide, 2014), extending particle filtering for Bayesian Inference. Other machine learning approaches use techniques that have both robustness and efficiency guarantees like GANs. Most relevant to this paper, Kaji, Manresa and Pouliot (2020) whose GAN approach to structural modelling open the door to robust and near efficient point estimation using only simulations from the model and not a likelihood.

There are a variety of ways to solve dynamic macroeconomic models: perturbation, projection, and value function iteration (Fernández-Villaverde, Rubio-Ramírez and Schorfheide, 2016), (Judd et al., 2017), of which only perturbation yields likelihood functions that allow Bayesian and MLE estimation using conventional techniques. Perturbation works well when the policy function looks like a linear or low degree polynomial function. For methods that are highly nonlinear, with large shocks (Terry, 2017), or even non-analytic, projection and value function iteration should be used. For a large set of models, almost anything that is solved via value function iteration (Hennessy and Whited, 2007), heterogeneous agent models with intractable likelihoods (Kukackaa and Barunika, n.d.), and most projection models the likelihood is intractable forcing the use of less efficient point estimate methods that have weaknesses compared to Bayesian and MLE estimation.

I will give an overview of current simulation based estimation and related approaches. Most of the simulation based likelihood and inference approaches are inspired by an approach like method of simulated moments (MSM) (McFadden, 1989), (Pakes and Pollard, 1989), (Duffie and Singleton, 1990). Due to the difficulty of verifying the global identification criterion there is a large interest in developing more efficient and robust estimators where this is not a problem. Techniques like maximum simulated likelihood, attempt to address this (Lerman and Manski, 1981), although these techniques both have simulation bias (Hajivassiliou et al., 1997), (Haan and Uhlendorff, 2006) and typically require the use of measurement error. There has also been work on using techniques from simulation based likelihood estimation to solve dynamic models, particularly in industrial

organization (Keane and Wolpin, 1994). There is also literature on efficient simulation techniques using both indirect inference (Smith Jr, 1993), (Gourieroux, Monfort and Renault, 1993), and efficient method of moments (Gallant and Tauchen, 1996).

In order to estimate non-perturbation models, one has to resort to simulation based inference approaches. As such, most of the time, these models are not estimated in a Bayesian or even full information manner and practitioners resort to, for example, method of simulated moments. This paper shows how to estimate likelihood-free small and medium sized models solved via value function iteration, among other methods, in a Bayesian fashion. The algorithm can even handle high dimensional output of HANK models with 821 different covariates per time step for simulated data, showing that the approach also works with high dimensional HANK models which are increasing becoming workhorse models in macro.

As an illustration of problems faced by a perturbation approach, below is an example of a s-S model policy function in Caballero and Engel (2007):

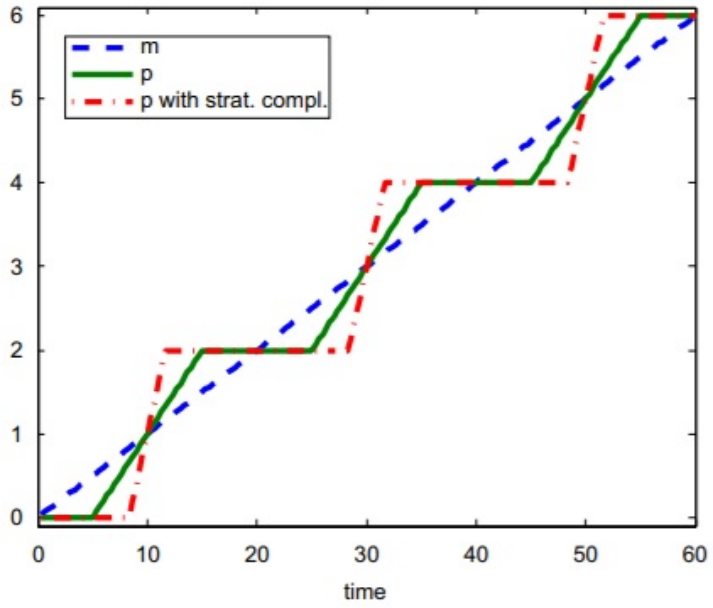

Fig. 2. p and m in an extension of Caplin–Spulber.

Figure I: s-S model

s-S models posits a fixed cost to changing a state variable which typically leads to kinks in the policy function. Often times modeling inventory or capital requires this assumption. The above example uses s-S costs with respect to menu prices assuming a linearly increasing money supply (blue). This is a modification of Caplin and Spulber (1987), which assumes there is a spectrum of firm prices from the s-S trigger point to zero. If the spectrum of firms prices only covers a portion of the range from the s-S trigger to zero, as the money supply increases linearly, at some time periods there will be no firms adjusting due to the fixed costs and at some time periods an above average number of firms will adjust to keep up with the price level, which is what the green and red line show. The kinks are where the policy function is non-differentiable and shows the transition where no one is adjusting to when someone is adjusting prices or vice versa. Thus a perturbation approach, which estimates a Taylor series approximation of the policy function, will not work as the policy function is non-analytic. This implies that one must solve the model via something like value function iteration, which leads to estimation problems mentioned above.

Papers which estimate s-S models include Arrow, Harris and Marschak (1951) and Caplin and Spulber (1987). In particular, Khan and Thomas (2008) assumes an s-S model for heterogeneous agent model for investment in capital. House and Leahy (2004) model the purchasing behavior for used durable goods given a fixed cost in addition to the price of the good. In particular, the good is priced under adverse selection (Akerlof, 1978). Likewise in finance there is a large literature estimating structural models (He, Whited and Guo, 2021), (Taylor, 2013). Like with s-S models, this literature often includes fixed adjustment costs or other approaches that yield kinks in the policy function. Terry, Whited and Zakolyukina (2022) finds a negative externality that comes regulation of public company disclosure. Although they find current disclosure regulations are low, if one is forced to disclose accurately, one will mi-invest so that the investments generate results that more closely resemble the the desired upon false disclosure results.

III. Background on Simulation Neural Posterior Estimation and Metropolis Hastings

This section will discuss the background of the SNPE estimator and MH-MCMC. First I will intuitively discuss the basics behind Bayesian estimation. Then I will move to a brief discussion of normalizing flows which is a density estimator used to estimate the posteriors from samples drawn from the joint distribution. The benefits of SNPE mainly stem from the use of the flow over traditional density estimators like the kernel density estimator (KDE). Finally I will conclude with a section on the SNPE algorithm.

When discussing the approach of simulation-based inference, it is useful to lay down a few definitions. The true data comes from the underlying data generating process. For macroeconomics, it would be economic data like output, consumption, etc. The simulator is the model being estimated, for this paper: a dynamic macro model. The only demand simulation-based inference puts on simulators is to be able to simulate data that has a 1-1 correspondence to the covariates in the true data.

III.A. Bayesian Basics

First I will discuss the general paradigm of Bayesian estimation, then I will discuss MH-MCMC.

Bayesian estimation attempts to find the posterior given the likelihood and prior of a model using Bayes' rule:

$$P(\theta|x) = \frac{P(x|\theta)P(\theta)}{P(x)}$$

Here $P(\theta|x)$ is the posterior, $P(x|\theta)$ is the likelihood, and $P(\theta)$ is the prior. The posterior is the probability of your parameters occurring after seeing the data and should be considered the optimal best distributional guess for the parameter values if ones belief before seeing the data was the prior. The likelihood is the probability the model used assigns to the data occurring conditional on θ. $P(\theta)$ is the prior of the model which is the probability the modeler assigns to the probability of theta, before seeing any data. $P(x)$ is the partition function, which is the main difficulty in Bayesian

inference as this requires solving an intractable integral. All techniques the perform Bayesian Inference, ranging from MH-MCMC, variational inference, to simulation-based inference concern techniques for calculating $P(\theta|x)$ without calculating the partition function.

The predominant technique for performing Bayesian Inference in economics is to perform MH-MCMC (Herbst and Schorfheide, 2015), which I will describe in this paragraph. In MH-MCMC, one only concerns oneself with relative jumps between parameter values. For example, if one knows both the prior and the likelihood for any given point, which is assumed to be known, one knows the relative likelihood of being in any point versus any other. One point in parameter space may be twice as likely to be visited in the posterior as another point, even if the actual probabilities are not known. Since one knows the relative probabilities, one can design a random walk so that a walker visits probabilities equivalent to the ratios of there probabilities. One way to do that is if a walker can choose to compare any point in the posterior to the point the walker is currently at, the walker will move to the second point with the likelihood according to the ratio of the probabilities. If the second point is half as likely to be in the posterior, the walker will move with probability one half and probability one half stay in place. If the second point is more likely to be in the posterior, say twice as likely, the walker will move to the second point for sure, with the understanding that if the walker randomly selected to move back, the move back would be now with half the probability.

In this way, the walker moves across the space of the posterior with relative frequency according to $P(x|\theta)P(\theta)$ ratio between points, which ultimately means that one is sampling from $P(\theta|x)$ without calculating the normalizing partition function. In practice, MCMC is a little more complicated, as often the parameter space is unbounded, and one can't sample the entire space with equal probability if the space is unbounded. Thus this involves the use of a proposal distribution and the use of importance sampling (see appendix for more information).

Alternatives like variational inference have been proposed which speed up the inference at the cost of some bias (Wainwright and Jordan, 2008), but it is beyond the scope of this paper. The approach I propose, simulation-based inference, does not require knowing the likelihood, something that may be impossible to define to begin with and also is faster than MH-MCMC in many situations.

III.B. Normalizing Flows

sectionNormalizing Flows In this section, I will discuss how a particular normalizing flow, the neural autoregressive flow, is structured. Then I will discuss how to use the change of variables formula for a random variable to derive a likelihood of a sample under the flow.

The main difficulty is to define the intermediate changes in measure, $f_a, f_b ... f_z$, so that they can be stacked to form a invertable function. One set of invertable transformations are affine operations. Given $y^i \in R^N$ and n going from 1..N indexing each element in the vector y^i_n:

$$\forall n, y^j_n = \beta^0_n + \beta^1_n * y^i_n \tag{1}$$

This is trivially invertable by shifting by the negative of β^0 and scaling by the inverse of β^1. While one can still add nonlinearities like a logit transformation between bijective couplings, this still lacks expressivity. The standard way to extend this model is to allow for the shifts and scales to be dependent on at least some of the inputs. For example, earlier elements $y^i_{a:l}$ are only modified in an directly invertable manner (ie $y^i_{a:l} = \beta^0 + \beta^1 * y^j_{a:l}$), then condition the shifts and scales for the rest of the elements $y^i_{l:N}$ on inputs, $y^i_{a:l}$, that weren't transformed:

$$\forall \iota \in 2 : N, y^j_\iota = \beta^0_\iota(y^i_{a:\iota-1}) + \beta^1_\iota(y^i_{a:\iota-1}) * y^i_\iota \tag{2}$$

$\beta^0(.)$ and $\beta^1(.)$ are typically differentiable and flexible functions–typically feed-forward neural networks conditioned on earlier elements of the vector, that now output dynamic parameters β^0, $beta^1$ that depend on the conditioning variables. Next I will illustrate how a neural autoregressive bijector is defined that I use in my paper. Make the first element, y^j_1 the identity or affine map, then the second element y^j_2 an affine transformation in y^i_2 with psuedo-parameters $\beta_0(y^i_1)$ and $\beta_1(y^i_1)$ dependent only on y^i_1. The third element, y^j_3, has psuedo-parameters then dependent only on $y^i_{1:2}$ and y^i_3 enters only in an affine manner.

$$y_\iota^j = \sigma(\beta_\iota^0(y_{a:\iota-1}^i) + \beta_\iota^1(y_{a:\iota-1}^i) * y_\iota^i)$$

To make this model nonlinear, one typically adds a link function σ which is usually a ReLU. A ReLU can be understood as two lines of different slope intersecting at zero. This kink is often enough to make neural networks universal approximators of continuous functions (Cybenko, 1989).

Since σ is invertable as well as the rest of the flow, one can recover the pull back of the bijector knowing only all elements in y^j[2]. Since each affine transformation is a function of variables that have an index smaller than the index in question, the Jacobian is lower triangular and the determinant is just the product on the diagonal.

Furthermore, if one stacks multiple bijections on top of one another, one can permute the order of input elements ($y_{1:N}^i$) with a bijective operation so that different bijectors have different conditioning relationships among variables allowing the flow to universally approximate any distribution. If the functions for $\beta^0(.)$ and $\beta^1(.)$ are universal approximators (ie neural networks), one can show that a stack of bijectors (along with permutations), is also a universal approximator and thus can approximate any change of variable arbitrarily well. This will be discussed and proved in the theoretical section in the appendix following (*Huanget al.*, 2018).

One can also condition $\beta^0(.)$ and $\beta^1(.)$ with additional arbitrary conditioning variables to form a flexible conditional density estimator. Since the problem is to estimate the density of θ conditioned on X, θ will take the role of $\{y_1^a..y_n^a\}$. X will be the true data and analogue of simulated data and in general x (lower case). Conditioning variable x form simulations or the real data will enter into the neural network that produces $\beta^0(.)$ and $\beta^1(.)$ and this approach will allow for conditioning. Thus, for example $beta_5^1$ will be conditioned on $y_1^i...y_4^i$ as well as x, the simulated data.

Since these models can model any conditional distribution, and can calculate the likelihood of any sample point in the target space, one can fit this model on samples from arbitrary continuous densities and have the guarantees that come with a well specified maximum likelihood problem.

2. The procedure to recover y_2^i from all the y_j's is to know y_1^i from the identity. Then recover the affine parameters for the y_2^i equation (which is only conditioned on y_1^i). Then one can invert y_2^j to get y_2^i knowing the affine parameters. Now that one knows $y_{1:2}^i$ one can repeat the procedure to get y_3^i, etc.

Thus, normalizing flows are density estimators that are robust to misspecification and asymptotically efficient due to estimation with maximum likelihood.

Given that normalizing flow is a neural network that stacks many invertable changes-in-measure, one can use the change of variable formula to derive the likelihood that a sample point was generated by the flow (Rezende and Mohamed, 2015). First one defines a base distribution–a known distribution like a multivariate normal. Then one uses the flow to transform the base to a target distribution. Since the likelihood of any point from a target distribution is known under the flow, one can perform gradient descent on the parameters of the flow to modify it in such a way that it learns a mapping from the base distribution to the target.

Next I will discuss the change of variable formula for a flow. Given a density, $p(y^a)$ with random vector realizations y^a; a normalizing flow is a function, $f(y^a)$ mapping y^a to a target random variable y^z, which has density, $q(y^z)$ such that the probability $q(y^z)$ is related to the probability $p(y^a)$ via:

$$q(y^z) = p(y^a)|det\frac{df^{-1}}{dy^a})| = p(y^z)|det\frac{df(y^a)}{dy^a}|^{-1} \tag{3}$$

In the appendix I show in more detail how to construct a flow so that it is both invertable and can approximate any continuous distribution. Assuming both these things, one can fit this density estimators on samples by using the change of variable formula to calculate the likelihood of any point, $q(y^z)$, and perform maximum likelihood.

III.C. Sequential Neural Posterior Estimation

First, I will discuss the procedure underlying Kristensen and Shin (2012), which is the same approach as the SNPE algorithm. Then I will discuss the role of the normalizing flow in extending the Kristensen and Shin (2012) approach as well as multi-round inference. I will first discuss the Kristensen and Shin (2012) approach to simulation inference. I proceed by simulating θ from the prior. Then one simulates x, simulated data from the model likelihood, from $P(x|\theta)$. Concatenating the two simulations together gives samples from the joint distribution $x, \theta \sim P(x, \theta)$. With these samples one can use the normalizing flow density estimator to estimate $P(\theta|x)$. Setting x to be the

real data X (note capitalized), allows one to have an estimate of the posterior.

One typically proceeds with SNPE in a multi-round fashion where a proposal distribution which has more alignment with the posterior takes the place of the sampling from the prior. Since the true posterior and the true prior might have limited overlap, to reduce variance, it's often important to estimate the posterior in multiple rounds. In the first round, one samples from the prior, in later rounds one samples from the current estimate of the posterior and performs importance samples to correct from the distribution shift. More mathematically, given a proposed prior of $p'(\theta)$, true prior of $p(\theta)$, the posterior, when trained on this data, will have to be importance sample adjusted by the factor of $\frac{p(\theta)}{p'(\theta)}$, to account for sampling from a distribution that isn't the prior. Greenberg, Nonnenmacher and Macke (2019) perform the estimation in one step by recognizing the adjusted distribution $p'(\theta|x) = p(\theta|x)\frac{p'(\theta)}{p(\theta)}$, where $p'(\theta|x)$ is the posterior obtained by sampling from a proposal distribution different then the prior. Then if a normalizing flow $f_\phi(\theta|x)$ is estimated in place of $p(\theta|x)$ in the above equation, a fully flexible $f_\phi(\theta|x)$ will return a unbiased estimator of the posterior. Importance sampling is more thoroughly discussed in the appendix.

III.D. Properties of SNPE

Before proving discussing properties of SNPE, I first prove a lemma that maximizing the likelihood is equivalent of minimizing the Kullback Leibler (KL) divergence between the model and the data distribution. **Lemma 9**: *Minimizing the KL divergence between the data, $P(x)$, and the model likelihood, $Q_\theta(x)$ is equivalent to maximizing the likelihood.*

$$\min_\theta KL(P(x)||Q_\theta(x)) := \min_\theta E_{P(x)}[log(P(x)) - log(Q_\theta(x)] \tag{4}$$

$$= \min_\theta -E_{P(x)}[log(Q_\theta(x)] \tag{5}$$

$$= \max_\theta E_{P(x)}[log(Q_\theta(x)] \tag{6}$$

The second line is equal to the third because $P(x)$ is not a function of θ. The other qualities are the result of algebra or definitions.

A brief discussion of the intuition of why SNPE converges to the Bayesian Posterior

Assumptions:

1. The unique parameters of the normalizing flow that generate the posterior is contained in the interior of possible parameter sets.

2. Continuous Assumption: The posterior produced, $P(\theta|x(\theta))$ is continuous functions of the parameters in the support of the model likelihood and prior almost everywhere

3. There exists a function $H(x)$ which upper bounds the likelihood and prior.

Since multi-round SNPE samples from a proposal distribution, $p'(\theta|x)$, then fitting the following function on joint simulated data will result in importance sampling corrected posterior: $f_\phi(\theta|x)\frac{p'(\theta|x)}{p(\theta)}$, where f_ϕ is the flow. See importance sampling discussion in appendix for details. Since the flow is performing MLE, it will pick parameters that that minimize the KL divergence between the flow and the correct posterior. The three assumptions are required for valid and unique MLE parameter estimation. Since the flow is a universal approximator (proof in appendix), given enough samples and a large enough flow, the true posterior will be arbitrarily close in a KL divergence sense to the best parameterization of the flow.

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(\theta|x)$;

end

IV. Results

Each model has it's own set of results. The first model is the RBC model where I estimate model on RBC simulated data and real data. The second model is toy corporate finance model solved via value function iteration and three different simulation algorithmic approaches are used to solve this model. The third model is a HANK model solved via Reiter's method and time iteration. Finally the forth model is the model of bequests which is solved via value function iteration. For more information on the details and construction of the models, refer to the appendix. All four methods have been estimated using my simulation-based inference approach.

IV.A. RBC Model

The first set of results deal with the RBC model. The RBC model has 4 parameters: α, β, δ, and ρ as defined in the economic models section of the appendix. The CRRA elasticity, γ is set to 2. The model is simulated for 200 iterations and the first 100 iterations is dropped to get a steady

state behavior. The results of the posterior with the parameters displayed in the same order as above is shown here:

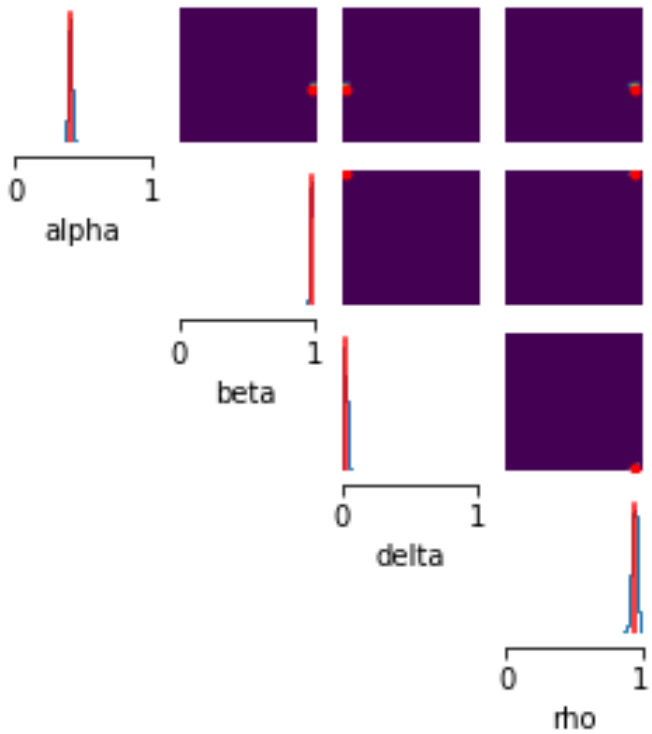

Figure II: Simulated Data RBC

The red bar indicates the true parameter values and the blue line indicates the posterior of the model as estimated by the simulation-based inference approach. Each of the density plots in the upper triangular portion of the graph indicates two way densities given by the corresponding row and column. Likewise the red dot indicates the true value of the parameter with respect to the two axis. In all models, uniform priors are used along the interval specified for all parameters.

As is clear, the simulation-based inference approach concentrates almost entirely on the true parameter value. This extremely close relationship is not entirely replicated on the other models, as the other models are are have flatter likelihoods and more parameters.

The second chart is the same RBC model estimated on real data:

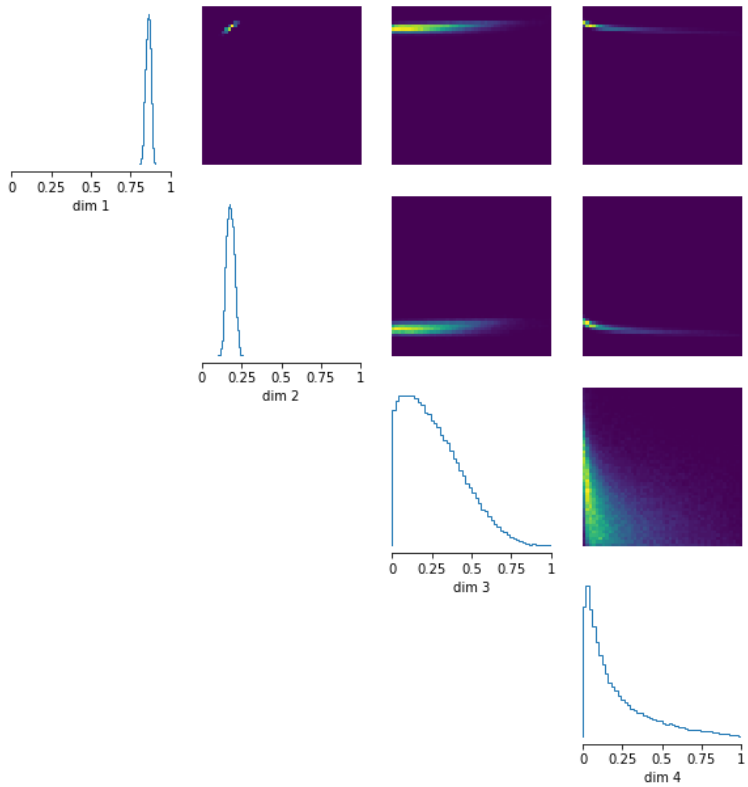

Figure III: Real Data RBC

Here there is no available ground truth but the parameters derived are not realistic, which is not surprising for such a simple model. You can see that compared to the first chart, the method is more uncertain of ground truth. You can also see the model doesn't estimate parameters close to what theory would suggest. This is not surprising given that the RBC model is too simple to work on real data and many of these parameters don't agree with theory even in more complex models. Furthermore to illustrate the accuracy of the model, all priors are uniform over the charted interval. In the model, it predicts the Cobb-Douglas parameter, α should be very close to 1, which is reasonable since labor is fixed in this model and capital is highly correlated with labor, which when combined usually implies a Cobbs-Douglas function that is homogeneous with degree 1. The discount rate β also doesn't agree with theory, which is unsurprising as beta is often calibrated

because it's so difficult to pin down. Likewise the parameter on the productivity process, ρ, is another variable that is often estimated but is also difficult to pin down–likely because different parameterizations have an important but subtle effect on estimation. This is displayed by the relatively uncertain posterior. δ or the depreciation rate, is the only parameter where the model is relatively reasonable.

IV.B. Cash Flow Model

The next set of results deal with a partial equilibrium model of firm cash flow solved via value function iteration. The parameters here are α, δ, σ, and ρ. σ is the standard deviation of the productivity law of motion. The interest rate is set at 5 percent and β is set as 1/(1+r). The cash flow model was estimated with three different methods, each of which obtaining nearly the same posterior, giving confidence the approach has converged to the right solution. The first chart will show the posterior for a simulation based inference approach that uses a normalizing flow and a feed forward network as the embedding network (converting the high dimensional x data into a lower dimensional conditioning variable). The second chart will show the same model and data, but using a RNN embedding network. The third chart will show a density estimator that is a GAN instead of a normalizing flow. This is an alternative simulation based estimator: Sequential Neural Ratio Estimation which is discussed in the appendix. Displaying the charts:

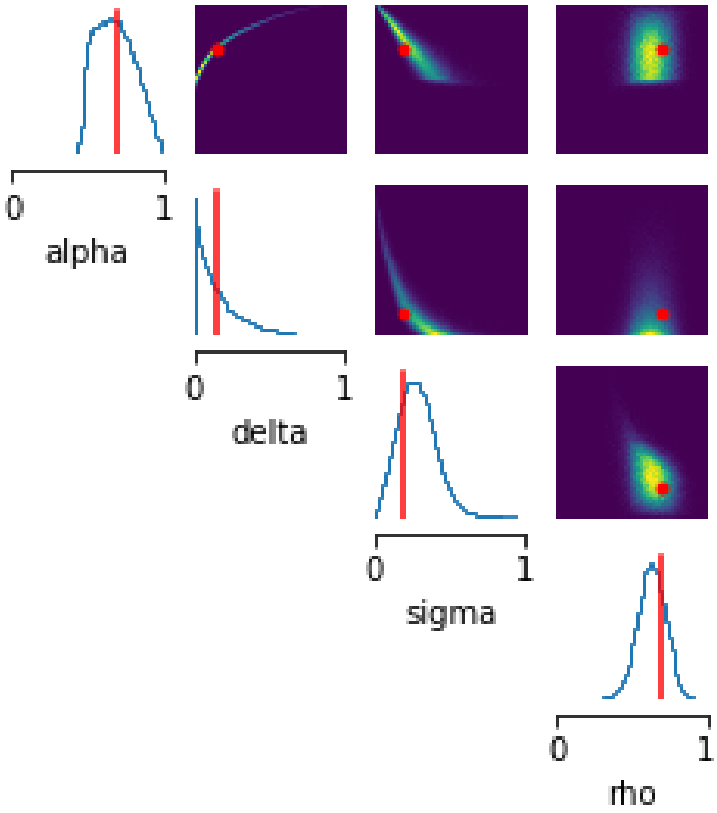

Figure IV: Value Function Iteration with a Dense Net Embedding

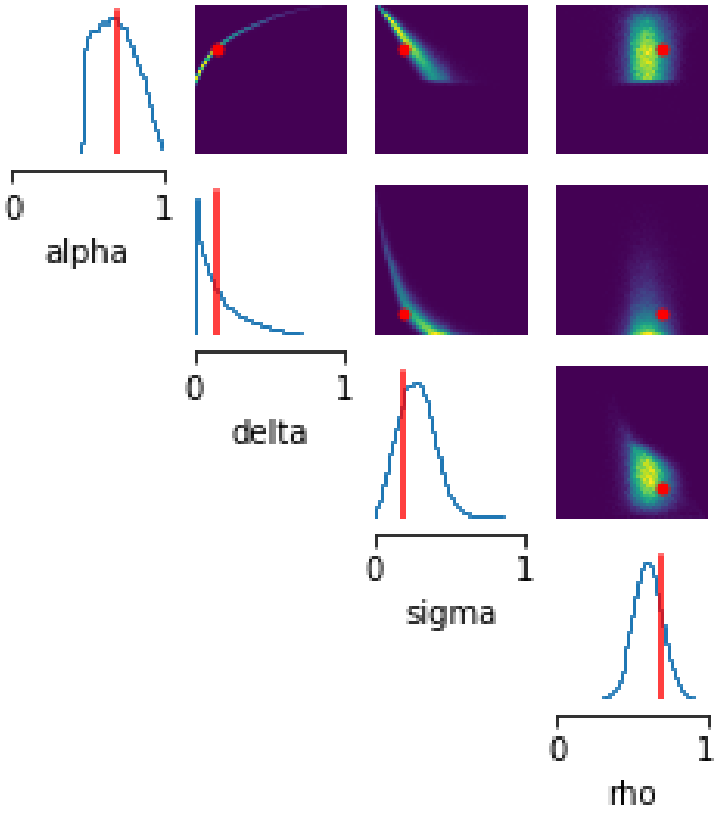

Figure V: Value Function Iteration with an RNN Embedding

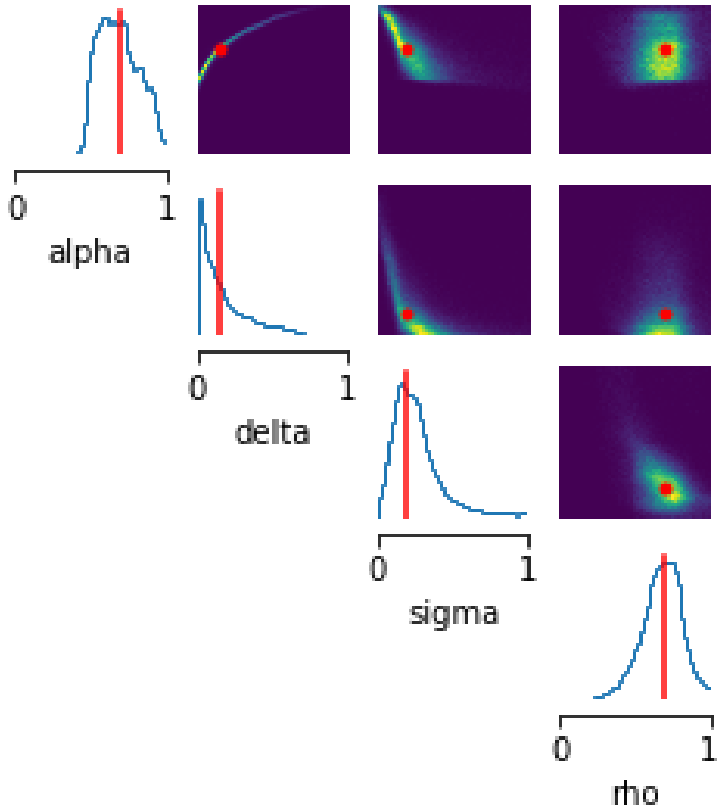

Figure VI: Value Function Iteration with a GAN Density Estimator

Despite using different inference techniques, the posteriors look fairly equivalent by visual inspection, suggesting the same distribution has been learned. Furthermore it does seem like the mode is fairly close to the actual parameterization. There some higher order considerations that are getting in the way of the dynamics of the model, but the model seems to be able to accurately recover the true parameters.

IV.C. Lucas Asset Pricing Projection Model

The following section shows the estimation procedure on a Lucas asset pricing model:

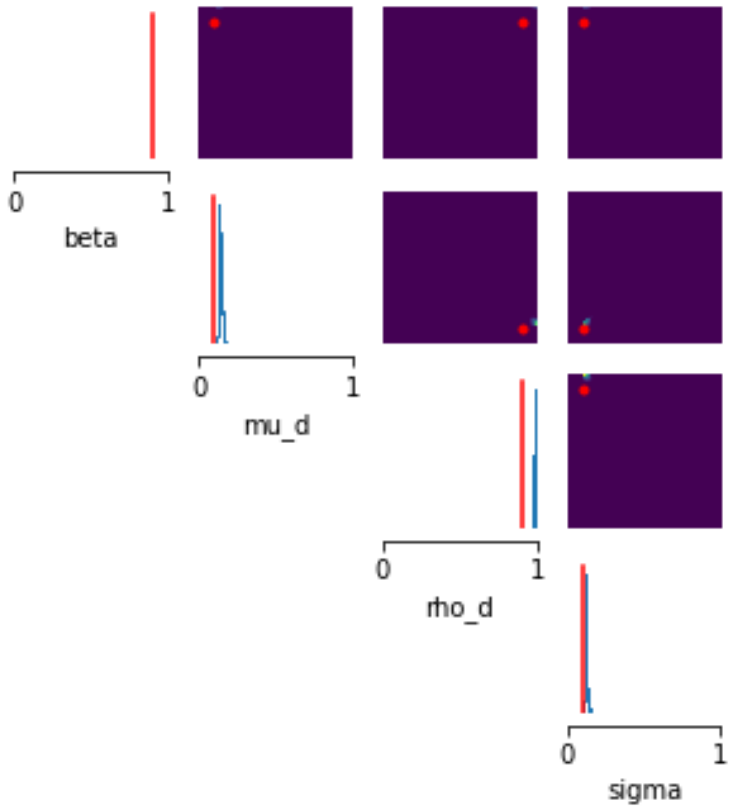

Figure VII: Ten Parameter HANK model

The model does seem to accurately fit reproduce the true values. the main error is on ρ_d where the posterior seems to be higher than the true value for the autogregressive parameters. All other posteriors match very closely the actual data generating parameters. This model took under 10 hours to estimate on an 8 core Intel i7 machine.

IV.D. HANK Model

The third model is the 10 parameter HANK model with the parameters discussed in the model section:

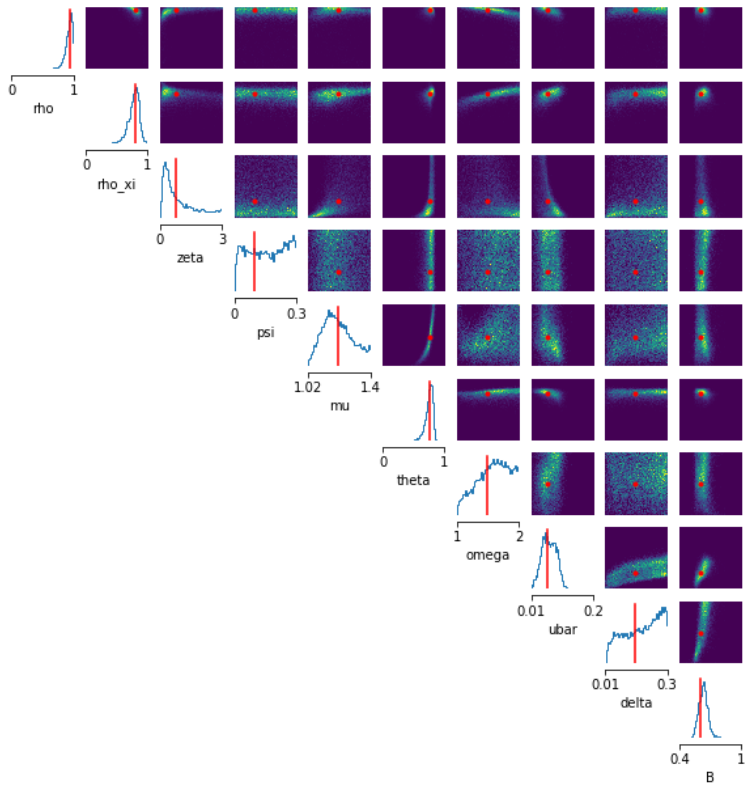

Figure VIII: Ten Parameter HANK model

It's impressive that the normalizing flow approach can estimate relatively formidable model especially one that has 61 timesteps and 829 covariates per time step. Here the model seems to recover the true parameter over most of the variables, but potentially due to the lack of time-steps, some parameters show flat likelihoods and modes in incorrect locations.

I will now discuss estimation speeds and comparisons with MH-MCMC. As mentioned in the introduction, this model took 57 hours to estimate on a 12 CPU Ryzen 7 5900X computer. The MH-MCMC result below took approximately 520 hours to estimate to generate 200,000 samples and is not yielding a usable posterior.

IV.E. HANK Model

The third model is the 10 parameter HANK model with the parameters discussed in the model section:

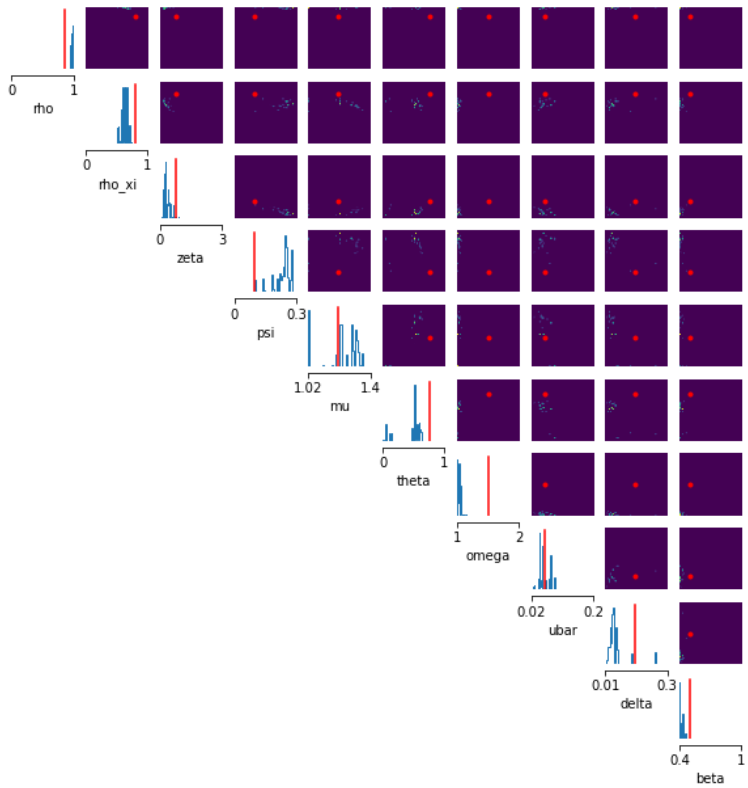

Figure IX: Ten Parameter HANK model - Estimated with MCMC

IV.F. Bequests Model

The fourth model is the 11 parameter bequests model with parameters displayed in the chart corresponding to the parameters in the model section:

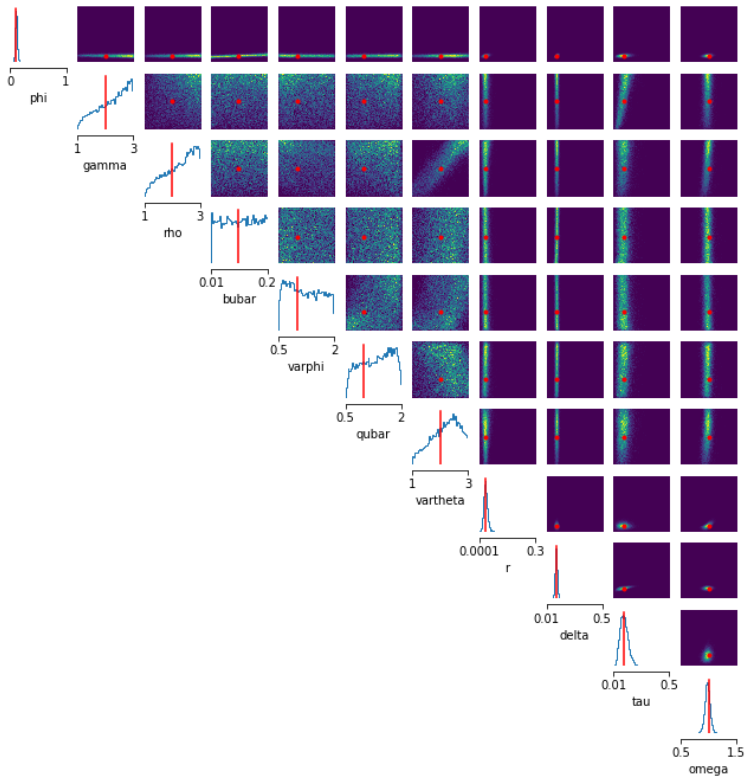

Figure X: Eleven Parameter Model Estimated via VFI

While many of the parameters have converged to their value of the true parameters, some of the parameters still have high uncertainty. This could be due to the fact that the parameters have very flat likelihoods. Regardless, this and the above model are research scale models, albeit on the small side. Many of the posterior distributions do concentrate around the true value, suggesting the model does have some discerning power with regard to the given parameters. Furthermore this is a reasonable sized model solved via value function iteration and the algorithm succeeds relatively well in calculating the posterior.

V. Conclusion

In this paper, I propose a method to estimate models in a Bayesian or Maximum Likelihood manner using simulations only, without a likelihood function. This works even on models that have a latent state space relationships across time. This approach allows estimation of dynamic macro models solved with value function iteration and projection methods in general with a Bayesian and MLE approach and can be applied to as many models as other simulation approaches like method of simulated moments. Additionally, outside of macroeconomics this approach can be applied to industrial organization, applied game theory, and econometrics. In industrial organization and game theory, this could be used to solve structural and dynamic models. In econometrics, general latent time series models can be estimated more effectively and efficiently, without likelihood functions. Additionally, the normalizing flow which is a bijective map that can perform arbitrary change of variables is a very powerful technique that hasn't been explored in depth in the field of econometrics. For example, normalizing flows can be applied to the identification in non-separable models literature and can be used as transport maps in optimal transport. A companion paper to this, focuses on using this technique to estimate general structural models on networks. Despite the focus on macroeconomic modeling, the application of this technique is much more general and has much greater applications that is beyond the scope of this paper.

References

Akerlof, George A. 1978. "The market for "lemons": Quality uncertainty and the market mechanism." In *Uncertainty in economics*. 235–251. Elsevier.

Altissimo, Filippo, and Antonio Mele. 2009. "Simulated non-parametric estimation of dynamic models." *The Review of Economic Studies*, 76(2): 413–450.

Arrow, Kenneth J, Theodore Harris, and Jacob Marschak. 1951. "Optimal inventory policy." *Econometrica: Journal of the Econometric Society*, 250–272.

Brehmer, Johann. 2021. "Simulation-based inference in particle physics." *Nature Reviews Physics*, 3(5): 305–305.

Caballero, Ricardo J, and Eduardo MRA Engel. 2007. "Price stickiness in Ss models: New interpretations of old results." *Journal of monetary economics*, 54: 100–121.

Caplin, Andrew S, and Daniel F Spulber. 1987. "Menu costs and the neutrality of money." *The Quarterly Journal of Economics*, 102(4): 703–725.

Carrasco, Marine, Mikhail Chernov, Jean-Pierre Florens, and Eric Ghysels. 2007. "Efficient estimation of general dynamic models with a continuum of moment conditions." *Journal of econometrics*, 140(2): 529–573.

Cranmer, Kyle, Johann Brehmer, and Gilles Louppe. 2020. "The frontier of simulation-based inference." *Proceedings of the National Academy of Sciences*, 117(48): 30055–30062.

Cybenko, George. 1989. "Approximation by superpositions of a sigmoidal function." *Mathematics of control, signals and systems*, 2(4): 303–314.

Dixit, Avinash K, and Joseph E Stiglitz. 1977. "Monopolistic competition and optimum product diversity." *The American economic review*, 67(3): 297–308.

Duffie, Darrell, and Kenneth J Singleton. 1990. "Simulated moments estimation of Markov models of asset prices."

Durkan, Conor, Iain Murray, and George Papamakarios. 2020. "On contrastive learning for likelihood-free inference." 2771–2781, PMLR.

Fernández-Villaverde, Jesús, and Juan F Rubio-Ramírez. 2007. "Estimating macroeconomic models: A likelihood approach." *The Review of Economic Studies*, 74(4): 1059–1087.

Fernández-Villaverde, Jesús, Juan Francisco Rubio-Ramírez, and Frank Schorfheide. 2016. "Solution and estimation methods for DSGE models." In *Handbook of macroeconomics*. Vol. 2, 527–724. Elsevier.

Flury, Thomas, and Neil Shephard. 2011. "Bayesian inference based only on simulated likelihood: particle filter analysis of dynamic economic models." *Econometric Theory*, 27(5): 933–956.

Gallant, A Ronald, and George Tauchen. 1996. "Which moments to match?" *Econometric theory*, 12(4): 657–681.

Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. "Generative adversarial nets." *Advances in neural information processing systems*, 27.

Gourieroux, Christian, Alain Monfort, and Eric Renault. 1993. "Indirect inference." *Journal of applied econometrics*, 8(S1): S85–S118.

Greenberg, David, Marcel Nonnenmacher, and Jakob Macke. 2019. "Automatic posterior transformation for likelihood-free inference." 2404–2414, PMLR.

Haan, Peter, and Arne Uhlendorff. 2006. "Estimation of multinomial logit models with unobserved heterogeneity using maximum simulated likelihood." *The Stata Journal*, 6(2): 229–245.

Hajivassiliou, Vassilis A, et al. 1997. *Some practical issues in maximum simulated likelihood.* Suntory-Toyota International Centre for Economics and Related Disciplines ....

He, Li, Toni M Whited, and Ran Guo. 2021. "Relative Performance Evaluation and Strategic Competition." *Available at SSRN 3287143*.

Hennessy, Christopher A, and Toni M Whited. 2007. "How costly is external financing? Evidence from a structural estimation." *The Journal of Finance*, 62(4): 1705–1745.

Herbst, Edward, and Frank Schorfheide. 2014. "Sequential Monte Carlo sampling for DSGE models." *Journal of Applied Econometrics*, 29(7): 1073–1098.

Herbst, Edward P, and Frank Schorfheide. 2015. *Bayesian estimation of DSGE models.* Princeton University Press.

Hermans, Joeri, Volodimir Begy, and Gilles Louppe. 2020. "Likelihood-free mcmc with amortized approximate ratio estimators." 4239–4248, PMLR.

House, Christopher L, and John V Leahy. 2004. "An sS model with adverse selection." *Journal of Political Economy*, 112(3): 581–614.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Hyvärinen, Aapo, and Petteri Pajunen. 1999. "Nonlinear independent component analysis: Existence and uniqueness results." *Neural networks*, 12(3): 429–439.

Judd, Kenneth L, Lilia Maliar, Serguei Maliar, and Inna Tsener. 2017. "How to solve dynamic stochastic models computing expectations just once." *Quantitative Economics*, 8(3): 851–893.

Kaji, Tetsuya, Elena Manresa, and Guillaume Pouliot. 2020. "An adversarial approach to structural estimation." *arXiv preprint arXiv:2007.06169*.

Keane, Michael P, and Kenneth I Wolpin. 1994. "The solution and estimation of discrete choice dynamic programming models by simulation and interpolation: Monte Carlo evidence." *the Review of economics and statistics*, 648–672.

Khan, Aubhik, and Julia K Thomas. 2008. "Idiosyncratic shocks and the role of nonconvexities in plant and aggregate investment dynamics." *Econometrica*, 76(2): 395–436.

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

Kukackaa, Jiri, and Jozef Barunika. n.d.. "Non-Parametric Simulated Maximum Likelihood Estimation of Parameters in Heterogeneous Agent ModelsI Preliminary Draft Version: May 19, 2015."

Lerman, Steven, and Charles Manski. 1981. "On the use of simulated frequencies to approximate choice probabilities." *Structural analysis of discrete data with econometric applications*, 10: 305–319.

Li, Yujia, Kevin Swersky, and Rich Zemel. 2015. "Generative moment matching networks." 1718–1727, PMLR.

Matzkin, Rosa L. 2003. "Nonparametric estimation of nonadditive random functions." *Econometrica*, 71(5): 1339–1375.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Pakes, Ariel, and David Pollard. 1989. "Simulation and the asymptotics of optimization estimators." *Econometrica: Journal of the Econometric Society*, 1027–1057.

Rezende, Danilo, and Shakir Mohamed. 2015. "Variational inference with normalizing flows." 1530–1538, PMLR.

Rubin, Donald B. 1984. "Bayesianly justifiable and relevant frequency calculations for the applied statistician." *The Annals of Statistics*, 1151–1172.

Smith Jr, Anthony A. 1993. "Estimating nonlinear time-series models using simulated vector autoregressions." *Journal of Applied Econometrics*, 8(S1): S63–S84.

Taylor, Lucian A. 2013. "CEO wage dynamics: Estimates from a learning model." *Journal of Financial Economics*, 108(1): 79–98.

Tejero-Cantero, Alvaro, Jan Boelts, Michael Deistler, Jan-Matthis Lueckmann, Conor Durkan, Pedro J. Gonçalves, David S. Greenberg, and Jakob H. Macke. 2020. "sbi: A toolkit for simulation-based inference." *Journal of Open Source Software*, 5(52): 2505.

Terry, Stephen J. 2017. "Alternative methods for solving heterogeneous firm models." *Journal of Money, Credit and Banking*, 49(6): 1081–1111.

Terry, Stephen J, Toni M Whited, and Anastasia A Zakolyukina. 2022. "Information versus investment." National Bureau of Economic Research.

Wainwright, Martin J, and Michael Irwin Jordan. 2008. *Graphical models, exponential families, and variational inference.* Now Publishers Inc.

A Bayesian Estimation in General and Metropolis-Hastings

The predominant technique for performing Bayesian Inference in economics is to perform MCMC (Herbst and Schorfheide, 2015). In MCMC, one only concerns oneself with relative jumps between parameter values. For example, if one knows both the prior and the likelihood for any given point, which is assumed to be known, one knows the relative likelihood of being in any point versus any other. One point in parameter space may be twice as likely to be visited in the posterior as another point, even if the actual probabilities are not known. Since one knows the relative probabilities, one can design a random walk so that a walker visits probabilities equivalent to the ratios of there probabilities. One way to do that is if a walker can choose to compare any point in the posterior to the point the walker is currently at, the walker will move to the second point with the likelihood according to the ratio of the probabilities. If the second point is half as likely to be in the posterior, the walker will move with probability one half and probability one half stay in place. If the second point is more likely to be in the posterior, say twice as likely, the walker will move to the second point for sure, with the understanding that if the walker randomly selected to move back, the move back would be now with half the probability.

In this way, the walker moves across the space of the posterior with relative frequency according to $P(x|\theta)P(\theta)$ ratio between points, which ultimately means that one is sampling from $P(\theta|x)$ without calculating the normalizing partition function. In practice, MCMC is a little more complicated, as often the parameter space is unbounded, and one can't sample the entire space with equal probability if the space is unbounded. In practice this involves the use of a proposal distribution and the use of importance sampling (see appendix for more information), which allows sampling from the entire space as long as one adjusts for the probability that the proposal distribution is

more likely to sample from one place than another by dividing by the likelihood of sampling from a particular point. In particular call $P(x|\theta)P(\theta)$ as the unnormalized posterior value for a point θ and $P(x|\theta')P(\theta')$ is the unnormalized posterior for a point θ'. Given a proposal distribution $f(\theta)$ which is the probability of visiting θ from θ' and $f(\theta')$ as the probability of visiting θ' from θ under this proposal distribution, then the equation that preserves the relative ratios and samples from the posterior will move to the new point if:

$$\frac{\frac{P(x|\theta)P(\theta)}{f(\theta)}}{\frac{P(x|\theta')P(\theta')}{f(\theta')}}$$

Since often the f distribution is assumed to be symmetric, the f terms drop out leaving the ratios of the unnormalized probabilities used to decide when to walk. Unsurprisingly this approaches is quite slow as one has to visit the whole space sequentially with higher probability points being visited many more times than lower probability points. Alternatives like variational inference have been proposed which speed up the inference at the cost of some bias (Wainwright and Jordan, 2008), but it is beyond the scope of this paper. The approach I propose, simulation-based inference, does not require knowing the likelihood, something that may be impossible to define to begin with and also is somewhat faster than MCMC in many situations.

B Economic Models

The models used to test these algorithms are disparate. I estimate a relatively standard RBC model via time iteration, a model of capital formation solved via value function iteration, a HANK model solved via Rieter's method with time iteration and lastly a 11 parameter bequests model estimated via value function iteration. I will now discuss each of these models sequentially.

BA. The RBC Model

This RBC model is standard and comes from Alisdair McKay's note on Heterogeneous agent models: `https://alisdairmckay.com/Notes/HetAgents/index.html`.

The utility function is a standard CRRA utility:

$$u(C_t) = E\sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{7}$$

The production function is standard Cobb-Douglas with productivity:

$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{8}$$

Where $\bar{L}$ is fixed labor and Z_t is productivity and K_t is capital which evolves according to a standard law of motion:

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{9}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{10}$$

This model is solved via time iteration.

BB. The Model of Capital Formation

This model is a straightforward infinite horizon partial equilibrium model with risk neutral managers choosing investment to maximize the present value of a stream of firm cash flows and comes from code written by Emil Lakkis and Toni Whited.

Utility/cash flows are give by this formula:

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{11}$$

Where K_t, Z_t, and I_t are the same as in the RBC model–capital, productivity and investment respectively. π is the profit function.

The capital law of motion is like the RBC model:

$$K_t = (1 - \delta)K_{t-1} + I_t \tag{12}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{13}$$

Profit is defined as:

$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{14}$$

This model shares similarities with the RBC model, with the main difference being the utility function and the value function solution method.

BC. Lucas Asset Pricing Projection Model

Each agent maximizes CRRA utility:

$$max_i E_t[\beta^t \frac{c_t^{1-\gamma} - 1}{1 - \gamma}$$

with γ set to 3. The law of motion is:

$$k_{t+1} = (1 + d_t/P_t)k_t - c_t/P_t$$

The state of the system is defined as dividends received plus price of the amount of assets owned:

$$m_{t+1} = (P_{t+1} + d_{t+1})k_{t+1}$$

Putting these three equations together one gets the pricing kernel:

$$p_t = \beta^t E_t[(\frac{c_{t+1}}{c_t})^{-\gamma}(d_{t+1} + p_{t+1})]$$

Dividends evolve under an $AR(1)$ process:

$$d_{t+1} = \mu_d + \rho_d * d_t + \epsilon$$

Where ϵ is a shock with standard deviation σ. When behaving optimally, consumption also equals the dividends received.

BD. The HANK Model

The HANK model also comes from Alisdair McKay's tutorial, which adds a heterogeneous distribution of wealth as a state variable for households combined with a New Keynesian backbone.

Preferences are still CRRA, like the RBC model. The firm side is now new Keynesian with intermediate goods produced via:

$$y_{j,t} = Z_t N_{j,t} \tag{15}$$

Here Z_t is aggregate productivity and N_t is labor employed by firm j producing variety $y_{j,t}$. The final good is produced via the Dixit-Stiglitz aggregator Dixit and Stiglitz (1977):

$$Y_t = (\int_0^1 {y_{j,t}}^{\frac{\epsilon - 1}{\epsilon}})^{\frac{\epsilon}{\epsilon - 1}} \tag{16}$$

Here ϵ is the elasticity of substitution between intermediates.

Output is defined as,

$Y_t = A_t \int_0^1 n_{j,t} dj = A_t N_t$

Where $n_{j,t}$ is the labor for the individual intermediate goods. Additionally from the accounting identity, output is also, $Y_t = C_t + \psi M_t H_t$

Where the second term is hiring costs.

Government and bonds are defined by the following nominal and real budget constraint equations:

$$\tau P_t(w_t + d_t)(1 - u_t) + P_t B = (1 + i_{t-1})P_{t-1}B + P_t b u_t \tag{17}$$

$$\tau(w_t + d_t)(1 - u_t) + B = R_t B + b u_t \tag{18}$$

Labor market is a standard McCall Search model:

$$N_t = (1 - \delta)N_{t-1} + H_t \tag{19}$$

$$1 - u_t = (1 - \delta)(1 - u_{t-1}) + H_t \tag{20}$$

$$M_t = \frac{H_t}{u_{t-1} + \delta N_{t-1}} \tag{21}$$

$$M_t = \frac{1 - u_t - (1 - \delta)(1 - u_{t-1})}{u_{t-1} + \delta(1 - u_{t-1})} \tag{22}$$

The household maximizes the CRRA utility like the RBC model. The households budget constraint in real money is:

$$a_t + C_t = R_t * a_{t-1} + earnings \tag{23}$$

Here a_t is savings and R_t is the interest rate as determined by the bond market clearing condition.

Earnings are given by:

$$\text{earnings} = \begin{cases} (1-\tau_t)(w_t + d_t) & \text{if employed} \\ b & \text{if unemployed,} \end{cases} \tag{24}$$

Here w_t is wages, d_t is dividend income and b is unemployment benefits.

The stochastic matrix governing employment, unemployment transitions is given by:

$$\begin{pmatrix} 1 - M_t & \delta \\ M_t & 1-\delta \end{pmatrix} \tag{25}$$

Again M_t is governed by the labor market equations above.

The firm solves the Dixit-Stiglitz cost problem:

$$y_{j,t} = \left(\frac{p_{j,t}}{P_t}\right)^{-\varepsilon} Y_t \tag{26}$$

with price index P_t given by

$$P_t^{1-\varepsilon} = \int_0^1 p_{j,t}^{1-\varepsilon} dj. \tag{27}$$

The Calvo pricing adjustment parameter θ implies that the intermediate firm chooses price $p*$ to maximize:

$$\mathbb{E}_t \sum_{s=t}^{\infty} \theta^{s-t} R_{t,s}^{-1} \left[\frac{p_t^*}{P_s} y_{j,s} - w_s n_{j,s} - \psi M_s h_{j,s}\right] \tag{28}$$

Here $n_{j,t}$ denotes employment at firm j in time t. $h_{j,t}$ denotes hiring and $R_{t,s}^{-1}$ denotes the real interest rate discounted s periods back to time t. This model is solved using Reiter's method in combination with time iteration.

BE. The Value Function Iteration Bequests Model

Finally this is a model of bequests over time. The model illustrates the benefit of my approach as it cannot be estimated by perturbation because of the kink at the intersection of the consumption versus adjusting illiquid savings intersection. Thus one has to use simulation based inference to estimate this model solved via value function iteration.

This model comes from Jeppe Druedahl's open source model example: `https://github.com/pkofod/vfi/blob/master/Fast%20VFI.ipynb`. The utility function is:

$$u(b_t, c_t) = \frac{[\phi(b_t + \underline{b})^{1-\gamma} + (1-\phi)c_t^{1-\gamma}]^{\frac{1-\rho}{1-\gamma}}}{1-\rho} \tag{29}$$

Here b_t is bequest or non-liquid savings. c_t is consummation. b, ϕ, γ, ρ are all parameters. The bequest utility function (end of life utility) is:

$$\nu(a_t, b_t) = \varphi \frac{(a_t + b_t + \underline{q})^{1-\vartheta}}{1-\vartheta} \tag{30}$$

Here a_t is liquid savings and q, θ, φ are all parameters. The value function is give by:

$$v_t(m_t, n_t, l_t) = \max\{v_t^{keep}(m_t, n_t, l_t), v_t^{adj}(m_t, n_t, l_t)\} \tag{31}$$

$$\text{s.t.} \tag{32}$$

$$x_t = m_t + (1-\tau)n_t \tag{33}$$

Here m_t is beginning of period liquid savings (analogous to end of period a_t) and n_t is beginning of period illiquid savings (analogous to end of period b_t). x_t is the pooled value of total liquid and

illiquid savings. l_t is labor income. The post-decision value function is:

$$w_t(a_t, b_t, l_t) = \nu(a_t, b_t), t = T \tag{34}$$

$$w_t(a_t, b_t, l_t) = \mathbb{E}_t \left[\max\{v_t^{keep}(m_{t+1}, n_{t+1}, l_{t+1}), v_t^{adj}(x_{t+1}, l_{t+1})\} \right], t < T \tag{35}$$

$$\text{s.t.} \tag{36}$$

$$l_{t+1} \sim F(l_t) \tag{37}$$

$$m_{t+1} = (1 + r_a)a_t + \omega l_{t+1} \tag{38}$$

$$n_{t+1} = (1 - \delta)b_t \tag{39}$$

$$x_{t+1} = m_{t+1} + (1 - \tau)n_{t+1} \tag{40}$$

Here labor income comes from a distribution $F(l_t)$. Likewise $r_a, \omega, \delta, \tau$ are all parameters. The keep value function is:

$$v_t^{keep}(m_t, n_t, l_t) = \max_{c_t \in [0, m_t]} u(n_t, c_t) + \beta w_t(a_t, b_t, l_t) \tag{41}$$

$$\text{s.t.} \tag{42}$$

$$a_t = m_t - c_t \tag{43}$$

$$b_t = n_t \tag{44}$$

The keep value function is the value function at a point t if the agent chooses to consume and not withdraw or deposit funds into the illiquid asset, b_t. The adjust value function is:

$$v_t^{adj}(x_t, l_t) = \max_{b_t \in [0, x_t]} v_t^{keep}(m_t, n_t, l_t) \tag{45}$$

$$\text{s.t.} \tag{46}$$

$$m_t = x_t - b_t \tag{47}$$

$$n_t = b_t \tag{48}$$

The adjust value function is the value of adjusting the illiquid asset. Notice the agent is unable to

consume this period. This model with it's nonlinear and kinked decision rule cannot be effectively modeled with a method like perturbation. Even so, I use my estimation technique to get a Bayesian posterior distribution.

C Background on Simulation Neural Ratio Estimators

CA. General Approach of Simulation Based Inference

The differences between the two simulation based inference approaches roughly centers around what conditional density estimator to use. Sequential Neural Ration Estimators (SNRE) uses a GAN and Sequential Neural Posterior Estimators (SNPE) use a normalizing flow. While one can sample a lot of points from the joint, an iterative approach where one fits the density estimator on sampled data then re-samples leads to lower variance. I will start discussing SNRE, by introducing the GAN

CB. Generative Adversarial Networks

The GAN is an algorithm that uses two competing models moving in tandem to generate data that matches data generated from the real world (Goodfellow et al., 2014). Below is a diagram describing the structure of a basic GAN:

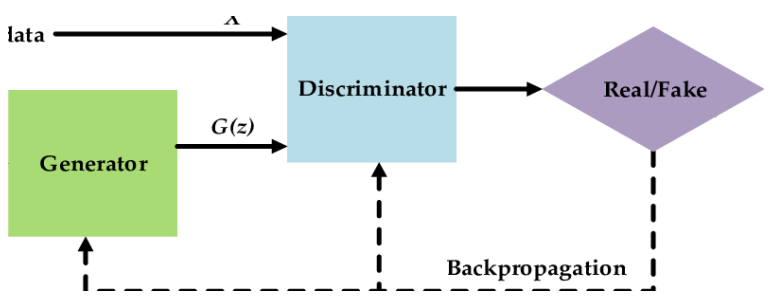

Figure XI: Diagram of a GAN

The generator is a neural network (or any differentiable model like a dynamic macro model) which attempts to produce data (unconditionally or conditionally via some inputs Z) that resembles

data in the real world. The discriminator is another neural network (or any trainable model that can produce a logistic output), which outputs the probability a given input data comes from the underlying data generating process and not from the generator. The two networks play a game where the objective of the generator is to convince the discriminator to assign high likelihood to its output and the objective of the discriminator is to assign the correct probabilistic label, a number between 0 or 1, to input that is either generated data or real data. One way to think about a GAN as a simulated method of moments algorithm where the moment conditions automatically evolve over many rounds to pick the most mismatched moments between the true data and the model. When thought in this way, it's intuitive to recognize that the GAN is nearly a full information and efficient estimation procedure. For more thorough proofs see Kaji, Manresa and Pouliot (2020).

More mathematically, the objective of the discriminator is to minimize a logit loss of input that comes either from real data x, or the generator $G(z)$ where the z are shocks that generate stochasticity in the GANs generation. Mathematically the discriminator loss is:

$$Loss_D = -E_{x \sim P_{data}(x)}[D(x)] - E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{49}$$

This is just a logit loss applied to data either coming from the data generating process or the generator. Likewise the generator's loss is to fool the discriminator:

$$Loss_G = E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{50}$$

These two loss functions can be compactly represented as:

$$G, D = \min_G \max_D E_{x \sim P_{data}(x)}[D(x)] + E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{51}$$

Equation (51) makes it clear the GAN objective is a minimax game with a solution that is a Nash equlibrium. However what is important for simulation based estimation is to recover likeli-

hoods/probabilities. The next proposition following Goodfellow et al. (2014) will show how to do that:

Proposition 1.: For a fixed generator, the optimal discriminator will produce a logistic output:

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{generator}(x)} \tag{52}$$

Proof: Using equation (49), for the loss of the generator:

$$Loss_D = \int_x log(D(x))p_{data}(x)dx + \int_z log(1 - D(G(z)))p_{generator}(z)dx \tag{53}$$

$$= \int_x (log(D(x))p_{data}(x) + log(1 - D(x))p_{generator}(x))dx \tag{54}$$

(55)

Any discriminator that maximizes the integral: $\int_x (log(D(x))p_{data}(x) + log(1 - D(x))p_{generator}(x))dx$, maximizes the integral pointwise over all x. Thus the optimal discriminator will set the first order condition of the intergrand to zero:

$$0 = \frac{p_{data}(x)}{D^*(x)} + \frac{p_{generator}(x)}{1 - D^*(x)} \tag{56}$$

Rearranging equation (56) gets the result of the proposition. ■

Then one can use this proposition along with the discriminator output to estimate a likelihood ratio.

CC. Sequential Neural Ratio Estimation

In SNRE, Durkan, Murray and Papamakarios (2020) use a GAN, not to match a underlying data like in proposition 1, but to contrast between two different distributions to estimate a density.

In macro, the dataset is typically a single data point with correlated timesteps. Using a GAN to discriminate between real data and fake data would not work because the GAN would learn a delta function for the real data. In order to ameliorate this, the authors use contrastive learning where they differentiate between the generated distribution and the independent distribution, $x, \theta \sim p(x)p(\theta)$, where x is sampled uniformly from the empirical distribution of simulated data. Then they contrast the independent distribution, with the true joint: $x, \theta \sim p(x, \theta) = p(x|\theta)p(\theta)$. Furthermore, knowing the optimal discriminator gives the ratio discussed in (52). The authors then recover the ratio: $r(x, \theta) = \dfrac{p(x, \theta)}{p(x)p(\theta)}$, using the fact that $\dfrac{p(x, \theta}{p(x, \theta) + p(x)p(\theta)}$ equals $\dfrac{r}{r+1}$. Thus, the density of the posterior $p(\theta|x) = r(x, \theta)p(\theta)$, where $p(\theta)$ is the prior. This extends Kaji, Manresa and Pouliot (2020) GAN approach in allowing estimation of models with a latent time dimension. One can extend this to multi-round inference where one uses the GAN as a better proposal distribution than the prior, by sampling θ from the GAN using MCMC. Then one uses importance sampling to deal with the mismatch in proposal distribution and prior. See Durkan, Murray and Papamakarios (2020), Hermans, Begy and Louppe (2020) and the SNRE theoretical sections of the appendix for more information.

Algorithm 2: SNRE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, discriminator $d_\phi(x,\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with MCMC;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x,\theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample

    $\{x'^{(k)}\}_k^{B*K}, \{\theta'^{(k)}\}_k^{B*K} \sim D\{x\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions x, θ and x', θ' and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(x^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(x^{(k)}, \theta^{(k)})})$;

  end

  Update posterior $p^{(i)} \propto exp(d_\phi(x,\theta))p(\theta)$;

end

CD. Properties of SNRE

Given three assumptions, a brief intuition of why SNRE converges to the Bayesian posterior:

1. The parameters of the discriminator, $f_\phi(\theta; x)$ that optimally discriminates true $p(\theta|x)/p(\theta)$ from any other distribution is unique and in the interior of the space of θ

2. The posterior produced, $P(\theta|x(\theta))$ continuous functions of the parameters and are derived from parameters in the support of the prior

3. There exists a function $H(x)$ which upper bounds the likelihood and prior.

Continuity along with the dominated convergence theorem using $H(x)$ allows one to say that

$sup_\theta|\hat{r}(\theta) - r(\theta)|$ converges in probability. Uniqueness and interior point establishes a global maximum that can be obtained in the parameterization of the set. The universal approximator theorem (Cybenko, 1989) illustrates that the entire space of continuous distributions is in the set and so given any almost everywhere continuous posterior and a large enough sample, the SNRE will be able to recover the ratio $r(x, \theta)$. When multiplied by the true prior or an importance sample adjusted proposal distribution (discussed in the proofs section of the appendix and in the article body) this gives a pdf of the true joint distribution $p(\theta|x) = r(x, \theta)p(\theta)$. Additionally, in multi-round sampling, the samples drawn from the proposal distribution via MCMC are distributed according the the unnormalized probabilities assigned to it by a GAN. Since by design, MCMC is able to correctly sample from unnormalized densities, this sampling procedure will be arbitrarily accurate given enough samples (Durkan, Murray and Papamakarios, 2020).

D Proofs

DA. Outline

There are two main parts of this section: 1.) How the Bayesian estimation procedure works, 2.) The estimation procedure with the normalizing flow density estimator.

DB. Bayesian Estimation

The Bayesian estimation procedure assumes the existence of a suitable density estimator that has all the properties of the normalizing flow. It will be convenient to assume the properties of the normalizing flow as stated which will be proved in the normalizing flow section (following Huang et al. (2018)).

The first step is to draw samples from the joint distribution and I will show this is possible. Since $P(x, \theta) = P(x|\theta)P(\theta)$, the best procedure is to draw θ from the prior, then draw x from the likelihood $P(x|\theta)$. This draws one sample (x, θ) from the joint distribution. Since the posterior is defined as $P(\theta|x)$, if one can fit a conditional density estimator on $P(\theta|x)$ on the joint samples

(x, θ) one can get the posterior distribution of θ by conditioning with the true data, X.

This procedure is enough to generate samples from the posterior distributions. However, this procedure has high variance if the prior is far away from the posterior as you are sampling θ in places where the posterior has little mass. A more computationally efficient way is to sample in multiple rounds. In the first round one uses the prior distribution, however in later rounds one samples $P(\theta)$ from a proposal distribution which should be the most updated guess on the posterior (denoted with a hat) $\hat{P}(\theta|x)$, then one adjusts via importance sampling for the fact that $\hat{P}(\theta|x)$ is not the same as $P(\theta)$.

Theorem 5 *In multi-round simulation based inference, estimating from a proposal distribution rather than a prior requires adding an adjustment factor to the density estimation of samples produced. Assuming the flow, $f(\theta|x)$, can converge to any probability distribution in distribution (Theorem 9), then when one fits $f(\theta|x) * \frac{\hat{P}(\theta)}{P(\theta)}$ to the data sampled from the proposal distribution $\hat{P}(\theta)$, one will converge to the true posterior $P(x|\theta)$*

Proof: In order to perform the adjustments note that $P(\theta|x) = P(x|\theta)P(\theta)/P(x)$. Call the incorrect proposal distribution $\hat{P}(\theta)$ and define $\hat{P}(\theta|x) = P(x|\theta)\hat{P}(\theta)/P(x)$ which is the conditional distribution that would be obtained by naively training a conditional estimator $f(\theta|x)$ on the joint that contains the incorrect proposal distribution. Then

$$\hat{P}(\theta|x) = (P(x|\theta)P(\theta)/P(x)) * \frac{\hat{P}(\theta)}{P(\theta)} = P(\theta|x) * \frac{\hat{P}(\theta)}{P(\theta)}(eq1)$$

Since the joint distribution with the differing proposal distribution has a pdf of $P(x|\theta)\hat{P}(\theta)$ so the conditional distribution which one is drawing samples from has pdf $\hat{P}(\theta|x)$. If one is fitting a density estimator $f(\theta|x)$ on the differing proposal joint data via maximum likelihood as one does with a normalizing flow, one will recover estimates the distribution $\hat{P}(\theta|x)$. However, if one fits the density estimator $f(\theta|x) * (\hat{P}(\theta)/P(\theta))$ (ie adjusting the likelihood by the importance weights), then $f_n(\theta|x)$ converges to $\hat{P}(\theta|x) * (P(\theta)/hatP(\theta))$ by the dominated convergence theo-

rem. $\hat{P}(\theta|x) * (P(\theta)/hatP(\theta)) = P(\theta|x)$ by eq 1. ■

In multi-round simulation-based inference, the prior is the first proposal distribution $\hat{P}(\theta) = P(\theta)$ and the normalizing flow is the resulting first round estimate of the posterior: $f_1(\theta|x = X)$. Then for the second round $\hat{P}(\theta) = f_1(\theta|x = X)$, where the proposal is now the estimated posterior recovered in the first round evaluated at the real data. Later rounds iteratively replace the proposal $\hat{P}(\theta) = f_{n-1}$ with the estimated posterior recovered from the previous round (and importance sample). Because importance sampling requires the ratio: $(\hat{P}(\theta)/P(\theta))$, f needs to be both easily sampled from as well as having easily calculable pdf. A normalizing flow can do this, as illustrated in the below section.

DC. Normalizing Flows in Depth

A normalizing flow is a complex model. I will discuss the constituent parts. Here is a decent description of them on an intuitive level: `http://akosiorek.github.io/ml/2018/04/03/norm_flows.html`. I encourage reading the blog post to understand flows at an intuitive level. This appendix will be more technical and so I would encourage consulting other sources for more intuition.

The point of a normalizing flow is to estimate the density from samples. The idea is to learn a mapping from an easy to model distribution like the multivariate normal distribution, to the target distribution which is learned via samples. The way this is done is to sample points from this target distribution and take the inverse mapping and get the respective base space points. Since the density of the target space distribution is known using a change of variable formula, one can calculate the density (or likelihood) the flow assigns to any sample point in the target space. Then one optimizes the parameters of the flow via maximize likelihood.

Before starting I will prove a couple lemmas:

Lemma 1: *Given a normalizing flow comprised of bijectors stacked one on top of each other. The formula of each bijector:*

$$y_i^j = w(y_1^i, ...y_{i-1}^i) * \sigma(\alpha(y_1^i, ...y_{i-1}^i) * y_i^i + \beta(y_1^i, ...y_{i-1}^i))$$

with α, β, and w being outputs from neural networks with all positive weights, monotonic non-linearities, σ, as well as all elements from the base distribution y^i are positive. This results in a monotonically increasing bijector in y_n^i for each n

Note: The last requirement that the base distribution is all positive is without loss of generally. One can make all base distributions positive then provide a final monotonic non-linearity that maps the positive half real line to the whole real line like a log transform.

Proof: To prove that a bijector parametrized in this way is monotonic, take a derivative of y_i^j with respect to y_i^i.

$$dy_n^j/dy_n^i = d\sigma(a)/da * da/dy_n^i$$

Where a corresponds to the equation inside of σ. Since $\sigma(.)$ is monotonic, $d\sigma(a)/da$ has a derivative greater than 0. Likewise $da/dy_i^i = \alpha(y_1^i, ...y_{i-1}^i)$ which by assumption is set to be positive with a correct composition of weight and non negative domain restrictions. Thus each dimension of the bijector is monotonic, making the entire bijector monotonic (increasing). A composition and sum of monotonically increasing functions is monotonically increasing so the entire normalizing flow is monotonically increasing, by induction. ■

Lemma 2: *If a set E in X has zero measure and a function $g(X)$ is Lipschitz, then $g(E)$ will also have zero measure*

Proof: Suppose $E \subset A$ has measure zero. Then cover A with a countably finite union of increasing compact sets in R^n such that:

$$A \subset \bigcup_{k=1}^{\infty} C_k.$$

Let E_k denote the intersection of E with C_k. Define a measure on R^n in the target space μ. If $\mu(g(E_k)) = 0$ for all k then $\mu(g(E)) = 0$ as a countable union of measure zero sets is measure zero. Since E_k has measure zero, for any $\delta > 0$ there is a cover of E_k with open balls $\{B(x_i, r_i)\}_{i=1}^m$ such that their total measure is less than or equal to δ. So,

$$E_k \subset \bigcup_{i=1}^{m} B(x_i, r_i)$$

If there is an $x \in E_k$ such that one of the m balls that covers E_k covers x: $|x - x_i| \leq r_i$. Thus if a function is Lipshitz $|y - g(x_i)| \leq \lambda_k r_i$ where λ_k is the Lipshitz constant. Thus

$$\mu(g(E_k)) \leq \mu\left(\bigcup_{i=1}^{m} B(g(x_i), \lambda_k r_i)\right) \leq \lambda_k^n \delta.$$

As the radius of the balls go to zero as δ goes to zero and the entire set is still covered. Thus $\mu(g(E_k)) = 0$, proving $\mu(g(E)) = 0$ ■

Lemma 3: *If a function J maps a random variable (the base distribution) X to another random variable Y (the target), and both random variables are subset of R^m, a sequences of functions J_n converges to J pointwise, then the random variables $Y_n = J_n(X)$ converges in distribution to $Y = J(X)$.*

Proof: (from (Huang et al., 2018)) Let h be any bounded continuous function on R^n. h composed with J_n converges pointwise to $h \circ J$ as compositions of continuous functions converge to their compositions. Since h is bounded, by the dominated convergence theorem $E[h(Y_n)] = E[h(J_n(X))] \to E[h(J(X))] = E[h(Y)]$. Since this result holds for any continuous function by the Portmanteau theorem this implies $Y_n \to Y$. ■

Now that these lemmas have been proven, I will discuss an individual bijector. I will first show why it is bijective in event space (ie samples) and then move to probability space and show why

it also is a valid change in measure and how to calculate the change of variables. Thus, if one knows the pdf of the base distribution, one also knows the distribution of the target distribution and vice versa. Lastly, I will discuss how to set up the model in a particular way, so that it is able to universally approximate any smooth bijective mapping between two random variables, resulting in being able to map any continuous base distribution to any continuous target distribution, which allows this density estimator to have a "non-parametric" universal approximator of distributions.

I will work with the autoregressive bijectors discussed up to this point:

$$y_i^j = w(y_1^i, ...y_{i-1}^i) * \sigma(\alpha(y_1^i, ..y_{i-1}^i) * y_i^i + \beta(y_1^i, ...y_{i-1}^i))$$

At this point, I will work in event space, so its not important to think about probability distributions but rather a mapping between two sets of numbers in R^n (ie samples from a multivariate distribution). A bijector will be shown to be a bijection and so the dimensionality between samples in the base space and the target space should be the same[3]. Call this n. The autoregressive flow orders the input and the output as so $y^i = \{y_1^i, y_2^i ... y_N^i\}, y^j = \{y_1^j, y_2^j ... y_N^j\}$. The name autoregressive comes from the fact that each output variable is only nonlinearly conditioned on input variables with a smaller index and y_1^j is just the identity of y_1^i. For example, the psuedo-parameters in the y_5^j equation is only a function of $y_1^i, y_2^i, y_3^i, y_4^i$. y_5^i only enters in an affine manner so is easily invertable. See the section III.B. for a more detail explanation of the architecture of a neural autoregressive normalizing flow.

After laying down the intuition, it is time to prove a bijector is a bijection.

Theorem 1: *A bijector learns a bijection between the input and output space.*

Proof: This will be done by proving three different elements are bijections and then the k sum of these elements are bijections. First $y^j = \alpha(y_1^i, ...y_{n-1}^i) * y_n^i + \beta(y_1^i, ...y_{n-1}^i)$ is a bijection. Then $\sigma(.)$ is a bijection and finally $y^j = w(y_1^i, ...y_{n-1}^i) * \sigma(.)$ is one. In order to prove that the first relationship is onto, first recognize the identity function is onto as $y_1^j = y_1^i$. Thus one can recover

3. This can be relaxed but is unimportant for this paper

y_1^i knowing y_1^j. Now proceed via induction. In order to get y_n^i from y_n^j assume one knows $y_{1:n-1}^i$. Thus given any y_n^i, α and β are fixed as a function of earlier $y_{1:n-1}^i$'s and thus are constant values with respect to y_n^i. Then given a y_n^j, one can recover a corresponding y_n^i that will produce it via the equation $(y_n^j - \beta)/\alpha$. To prove that this function is 1-1 proceeds in the same way. The identity is 1-1 and if $y_n^j = {y'}_n^j$, then $y_n^i = {y'}_n^i$, again by the fact that affine functions are 1-1, α and β are constants if one knows all previous elements of y^i. Then one extends this to all dimensions with induction in the same way as onto was proved. Proving $\sigma(y')$ is a bijection is by definition. σ is chosen to be a bijective nonlinearity. Proving the final step, $y^j = w(y_1^i, ...y_{n-1}^i) * \sigma(.)$, as this again is linear function in $\sigma(.)$ which is a subset of affine functions and in the first set of equations, I proved that affine functions are bijections, thus linear functions are bijections too. The final step involves proving the bijectiveness of y_i^j is made up of a sum of k of these equations. However, it is not true that a sum of bijections is a bijections. For example take $f = x$ and $g = -x$. One condition for a sum of bijections to be a bijection is for the bijections to be monotonic. In lemma 1, I prove that if the weights on the neural network in a neural autoregressive bijector are all positive and that the input (e.g. y^i as opposed to y^j) distribution is positive, this will make the model monotonic. Proceeding, given two monotonic continuous bijectors f and g, the sum is onto. Define h as $f + g$. h is continuous as a sum of continuous functions is continuous. Given a value y in the interior of the range of h there exists a value of $h|_v > y$ and a value of $h|_w < y$. Since h is continuous, by the intermediate value theorem it takes on a value of y. Likewise to prove 1-1, h is also monotonic as a sum of monotonic functions is monotonic. Thus, it can only take on that value of y exactly once. One can extend the proof of two monotonic bijections to arbitrary monotonic bijections by using induction. Combining the neural autoregressive bijector with a bijector that permutes the indices of y_n^i allows for arbitrary conditioning relationships. Now with a stack of bijectors, where the output of one bijector is the input of another, this is also a bijection as a composition of bijections is a bijections. Thus a normalizing flow is a bijection. ■

Proving bijectiveness, now I will work in the underlying density space/the space of measures and move away from just viewing the flow as a mapping of realizations of random variables. I will

now proceed to show that a normalizing flow is C-Lipschitz when the total domain is restricted to a set which has probability mass $1-\delta$ for arbitrarily small delta. This will allow me to show that the input of a normalizing flow absolutely continuous with respect to the output and vice versa. This is Lipsitz bounded by a constant $(\max_{|\alpha|,|\beta|}|\alpha|+|\beta|)^N$. Define the $max_{|\alpha|,|\beta|}$ of α and β as the maximum value these psuedo-parameters take on the compact set $V(1-\delta)$. The boundedness of the normalizing flow is just $(\max_{|\alpha|,|\beta|}|\alpha|+|\beta|)^N$ because each normalizing flow defines an affine relationship $\alpha(y_1^i,\dots y_{n-1}^i)*y_n^i+\beta(y_1^i,\dots y_{n-1}^i)$, which is bounded itself. Then, $\max_{\alpha,\beta}\alpha+\beta$ is the largest value of any α and β all the bijectors of the compact set take. Raising it the the power N corresponds to the number of times the bijector is composed with itself. Thus, the Lipschitz constant cannot be greater than this number. Likewise the reverse direction finds the max for the inverse values for alpha. This implies that the function is C-Lipschitz over any particular compact set that includes our data up to probability δ.

Theorem 3: *The chain of neural autoregressive bijectors along with a bijector that permutes the indices of inputs, learns a valid change in measure of an input base distribution to an output target distribution and the pdf of the target distribution can be derived using the standard change of variable formula:* $q(y^z)=q(y^a)|det(df/dy^a)|^{-1}$

Proof: I will now use Lemma 2 which proved that any Lipschitz functions preserves sets with measure zero. This implies that measures of both spaces are absolutely continuous with respect to one another. Additionally Lemma 3 which proves that if a sequence of flows f_n converges to f point-wise, the mapping of random variables $f_n(X)$ converges to $f(X)=Y$. Thus in both directions there exist a Radon Nikodym derivative and the Radon Nikodym derivative of the transformation of one random variable transformed by a bijector follows the convenient change of variables formula: $q(y^z)=p(y^a=f^{-1}(y^z))|det(df^{-1}/dy^a)|$ evaluated at a point y^z. where f maps y^a to y^z $(f(y^a)=y^z)$. This can be even more conveniently expressed as: $q(y^a)|det(df^{-1}/dy^a)|=p(y^a)|det(df/dy^a)|^{-1}$ This relationship can be shown by using the inverse function theorem and the recognition that the determinant of an inverse is the inverse of the determinant: $|det(df^{-1}/dy^a)|=|det((df/dy^a)^{-1})|=$

$|det((df/dy^a))^{-1}| = |det(df/dy^a)|^{-1}$, with derivatives all evaluated at a point y^a.

Now I have shown given a sample from y^z and a given distribution like y^a, how to recover the sample points in $p(y^a)$ that correspond to the points $q(y^z)$. First one inverts the bijector to get y^a. Then one can evaluate the base distribution on y^a. Finally, to calculate the likelihood of $q(y^z)$, one uses the change of variables formula to get the pdf of y^z under the bijector. One then can optimize the bijector using maximum likelihood and $q(y^z)$ as the likelihood of y^z.

A normalizing flow is just a stack of bijectors on top of one another so the output of one bijector becomes the input of the next bijector. Composition of bijections are also bijections and is invertible. Likewise a composition of Lipschitz functions is still Lipschitz and so this flow also is a valid change of measure. This proves that a normalizing flow, which is a stack of bijectors, is a valid change of measure whose density can be evaluated using the Radom Nikodym formula: $q(y^z) = p(y^a)|det(df/dy^a)|^{-1}$. ■

Finally I will show that a certain construction of normalizing flows are universal approximators of multivariate real probability distributions. This means the give a suitably chosen set of parameters the normalizing flow will converge in distribution to any continuous probability measure. In the proof, mirroring universal approximation proofs in general, I will focus on summing up an infinite number of bijectors, proving universal approximator properties for extremely wide normalizing flows, even though in practice deep normalizing flows are more often used.

Since the variables are conditioned only on previously indexed variables, of course a sum of bijectors all with the same conditioning structure between variables cannot approximate any dependency relationships between a target variable and a base variable. In practice this can be avoided by summing or stacking multiple bijectors and permuting the indices of the input, changing which variables are conditioned on which other variables.

DD. Normalizing Flows are Universal Approximators

The proof that normalizing flows are universal approximators will proceed in a series of steps following Huang et al. (2018). Initially, I will show that the flow can approximate any monotonic

function. The first sub-step is to show show that a sum of step functions are universal approximators of monotonic functions. Then I will show that if we make the nonlinearity σ a logit transformation, one can modify the logit transform to be arbitrarily close to a given step function. Then the proof will show that a neural network that approximates the optimal parameters w, α, β of the logit function can universally approximate any monotonic function. Finally, any continuous probability distribution can be mapped to any other probability distribution with a monotonic function. This will be made formal by showing monotonic functions can map any pdf to and from the multivariate uniform distribution.

In lemma 4-6 the domain and range of the two functions are between two arbitrary constants (r_0, r_1)

Lemma 4: *Step functions universally approximate monotonic functions*

Define:

$$S^*(y) = \sum_k w_k * s(y - b_j)$$

Where s is the step function that steps at $y = b_j$

Then the lemma states that for any continuous and monotonically increasing function $S : [a, b] \to [0, 1]$ and given any $\epsilon > 0$ there exists an $S^*(n)$ for some n such that $|S^*(y) - S(y)| < \epsilon \forall y \in [a, b]$. Abbreviating $s_j(y) = s(y - b_j)$, choose an $n = \lceil 1/\epsilon \rceil$ with the ceiling function and divide the range $[0, 1]$ into n+1 evenly spaced intervals: $[0, z_1), [z_1, z_2) ... [z_n, 1]$. For each z_j, there is a corresponding y_j value that corresponds to $y_j = S^{*-1}(z_j)$, since S^* is strictly monotonic. Now find the step function so all the $S^*(y_j) = z_j$, which will mean the error is at most ϵ as the difference in z_j is at most ϵ. To do this ensure that $s_j(y)$ steps at y_j by setting $b_j = y_j$. Then set the weights w_j so that w_j makes up the difference between z_{j-1} and z_j. This is equivalent to solving $Sw = t$ where S is the lower triangular matrix of ones that represents which steps have activated and t is the vector

of $[0, z_1, z_2 ... z_n, 1]$. The distance between $t_j - t_{j-1} = 1/(n+1)$, thus:

$$|S*(y) - S(y)| = |\sum_j s_j(y)(t_j - t_{j-1}) - S(y)| \tag{57}$$

$$= |1/(n+1) * \sum_j s_j(y) - S(y)| \tag{58}$$

$$\leq 1/(n+1) < \epsilon \tag{59}$$

■

Lemma 5: *I will move on to proving the same thing with sigmoid functions of the form*:

$$S'(y) = \sum_k w_k * \sigma((y - b_k)/\tau_k)$$

also can universally approximate arbitrary monotonic functions.

Note this can easily be translated into a form similar to our bijector by setting $\alpha_k = 1/\tau_k$, $beta_k = b_k/\tau_k$, and $w_k = w_k$. Note at this point α, β and w are constants and not functions. I will deal with making them functions in the next lemma. For any monotonic function, approximate the function using a step function up to a factor of $\epsilon/3$, using the previous lemma. Define $S'(y)$ such that $w_k = w_j$ and $b_k = b_j$ where the j index refers to the step function parameters ($S*$) and the k index refers to the parameters in the sum of sigmoids (S'). Then choose $\tau = \tau_k = \dfrac{\kappa}{\sigma^{-1}(1 - 1/(n-1))}$ where $\kappa = \min_{l \neq l'} b_l - b_{l'}$. Then $|\sigma((y - b_k)/\tau) - s(y - b_k)|$ is at most .5 if $b_{k-1} < y < b_{k+1}$ as the logit is around .5 right at b_k where the step steps from 0 to 1 and since the size of the steps are of magnitude $\epsilon/3$, the error is at most $.5 * \epsilon/3$. However, if y is not in this range then $sigma(y - b_k/\tau) = \sigma(y - b_k/(\min_{l \neq l'} b_l - b_{l'}) * \sigma^{-1}(1 - 1/(n-1))$ which implies that the distance from the step is at most $1/(n-1)$ for a 0 to 1 step function. Putting this together you have $\epsilon/3$ from the sum of the two one halves, then $\epsilon/3 = (n-1)/(n-1) * \epsilon/3$ from each other sigmoid that is not within $b_{k-1} < y < b_{k+1}$, which gives $2/3\epsilon$ from the logit approximation to the step function and

$\epsilon/3$ from the step function approximation to the monotonic function. Using the triangle inequality: $|S'(y) - S(y)| < |S'(y) - S*(y)| + |S*(y) - S(y)| < 2/3\epsilon + \epsilon/3 = \epsilon$ ■

Lemma 6: *Let the base and target space be multivariate and α, β, and w now be the outputs of neural networks conditioned on previous data. Bijectors parameterized like this*:

$$y_n^j = \sum_k w_k(y_1^i, ...y_{n-1}) * \sigma(\alpha_k(y_1^i, ...y_{n-1}) * y_n^i + \beta_k(y_1^i, ...y_{n-1})) \tag{60}$$

universally approximates monotonic functions.

I will prove this theorem by writing the bijectors in this form:

$$y_n^j = \sum_k w_k(y_1^i, ...y_{n-1}^i) * \sigma(\frac{y_i^i - b_k(y_1^i, ...y_{n-1}^i)}{\tau_k(y_1^i, ...y_{n-1}^i)}) \tag{61}$$

Since n indexes all the outputs of the bijector, I will proceed by proving the theorem for univariate case for each $y_n^j(y_1^i...y_n^i)$ and then extending the theorem for the multivariate function (eg the distance all n have from a multivariate monotonic function is smaller than a given ϵ) is relatively easy.

Proving this theorem requires showing there is a sequence of functions $y_n^j(y_1^i..y_n^i)$ such there exists a $k < K$ summed bijectors in $w_k(y_1, ...y_{i-1})$, $b_k(y_1, ...y_{i-1})$, and $\tau_k(y_1^i, ...y_{n-1}^i)$, that the error $|y_n^j(y_1^i...y_n^i) - S(y_n)|$ is smaller than any $\epsilon > 0$. Here y^j is the bijector and S is the monotonic function that needs to be approximated.

Since the fixed parameters can approximate any monotonic function arbitrarily well, I have from lemma 5:

$$|S'(y_n^i) - S(y_n^i)| < \epsilon/2$$

I also know from Cybenko (1989), that a wide enough neural network can approximate any function. Choosing a function that matches the parameters $S'(y_i)$ with low enough error, I know

there exists a neural network for the parameters $\alpha_k(.)$, $w_k(.)$, and $\tau_k(.)$ in y_n^i such that:

$$|y_n^j(y_1^i ... y_n^i) - S'(y_n^i)| < \epsilon/2$$

Now this distance bound is only true because the bijector function y^j is a bounded and continuous and so you can use the dominated convergence theorem to turn the the convergence of the sequence of parameters (e.g. $\lim_{l\to\infty} \alpha_k^l(.) \to \alpha_k$ etc), to a statement about the convergence of a sequence of functions: $\lim_{l\to\infty}\{y_n^j\}^l \to S'(x_n)$

Then the proof in the univariate case is finished by using the triangle inequality:

$$|y_n^j(y_1 ... y_n) - S(y_n^i)| < |y_n^j(y_1 ... y_n) - S'(y_n^i)| + |S'(y_n) - S(y_n)| \tag{62}$$

$$= \epsilon/2 + \epsilon/2 = \epsilon \tag{63}$$

The proof in the multivariate case is completed by recognizing that for each univariate case there is a K_i number of summed bijectors such that when $k_i > K_i$, $|y_n^j(y_1 ... y_n) - S(y_n^i)| < \epsilon$. Then just choose $k = \max_{k_i}$ and then $||y^j(.) - S(y_i)||_\infty < \epsilon$ ■

Now that I've demonstrated that a single very wide collection of bijectors is enough to approximate any monotonic function, it's clear stacking many wide bijectors can only improve model flexibility for a given width k. What is left to show is that monotone functions are enough to allow a normalizing flow to map any continuous base distribution to any continuous target distribution. The proofs will show that any distribution can be mapped to a multivariate uniform distribution with a normalizing flow. Likewise any distribution can be mapped from a uniform distribution. Since both are true, then it is clear that a flow can map any distribution to any distribution. These proofs are similar in spirit to Sklar's theorem for copulas, as copulas share similar proprieties as normalizing flows. The proof is complementary to econometric proofs for identification in non-separable models in the sense that it can in some non-rigorous way to be considered an existence proof to Matzkin (2003) partial uniqueness identification argument.

Lemma 7:Normalizing flows transform multivariate uniform random variables to arbitrary random variables. *The neural network parametrized by*

$$G(y^i)_t = \sigma^{-1}(\sum_k w_k(y_1^i, ...y_{n-1}^i) * \sigma(\alpha_k(y_1^i, ...y_{n-1}^i) * y_n^i + \beta_k(y_1, ...y_{n-1})))$$

can transform a base distribution ($Y \sim Unif((0,1)^m)$ of an m dimensional uniform random variables to any desired m dimensional random variable. Call this target distribution $Z \in R^m$. This convergence is in distribution to the desired random variable

This equation for G_n is the same equation as in lemma 6, with σ being a sigmoid/logit distribution. The one exception is the addition of the inverse sigmoid.

Proof: Let Z_t be a random vector in R^m and assume Y has strictly positive and continuous density. Take Y as the m dimensional uniform. Then there is a sequence of functions $\sigma^{-1} \circ \{y^z\}_t(y^a)$ where $\{y^z\}_t$ is the normalizing flow as discussed in lemma 6. Given an arbitrary ordering of the multiple CDFs of Z^t, defined as $F_t(Z_m < Z_m|y_1...y_{m-1})$, according to theorem 1 in Hyvärinen and Pajunen (1999), $F(Z)$ is distributed uniformly in the m dimensions and Z is independent of the y's. Then the inverse CDF function G_t is a monotonic function, as is $\sigma \circ G_t$. $G_t(F_t(Z_m < z_m|y_{1:m-1})|y_{1:m-1}) = Z_m$. Multivariate CDFs have an upper triangle dependency relationship. G_t is mapping from F_t which is a uniform distribution to the realizations of the random variables y_t. This by definition is a monotonic mapping. By lemma 6, $\sigma \circ G_t$, can be approximated arbitrarily well by a sequence of bijectors, $\{y^z\}_t(y^a)$. The σ is necessary because the preimage of $\{y^z\}_t(y^a)$ in lemma 6 is (r_0, r_1), which is only a bounded subspace of R^n. Thus, $G_t = \sigma^{-1} \circ \{y^z\}_t(y^a)$ converges uniformly, thus pointwise, to G, as σ^{-1} is continuous. Since G_t converges to G and $G(Y) = Z$, by lemma 2 we have $G_n \to Z$ in distribution. ■

Lemma 8: Given a correct parameterization, a normalizing flow that maps from arbitrary

distributions can converge in distribution to an multivariate uniform (0,1) distribution. *Let Y be a positive and continuous random vector in R^m. Let Z be distributed according to the multivariate uniform distribution. The there exists a sequence of functions $\{y^z\}_t(y^a)$ parameterized as in lemma 6, that converges in distribution to Z.*

Proof: Define the CDF for all y dimensions as: $z_1 = F_1(y_1|0)$ and $z_m = F_t(y_m|z_1...z_{m-1})$. To be clear $y_t = Pr(Y_m < y_m|z_1...z_{m-1})$, which again is a multivariate CDF with an upper triangular form. Again due to the theorem in Hyvärinen and Pajunen (1999), the distribution of the CDF is an n dimensional uniformly distribution. Then since lemma 6 shows that there exists a monotonic function, $\{y^z\}_t(y^a)$ that converges uniformly, thus pointwise, to F and $F(Y) = Z$, we have $\{y^z\}_t \to Z$ in distribution. ■

Theorem 9: *Bijectors in the form,*

$$H(y)_t = \sigma^{-1}(\sum_k w_k(y_1^i, ...y_{n-1}^i) * \sigma(\alpha_k(y_1^i, ...y_{n-1}^i) * y_n^i + \beta_k(y_1^i, ...y_{n-1}^i)))$$

can map arbitrary distributions to arbitrary distributions arbitrarily closely in distribution

Proof: The intuition here is to map any distribution to a uniform distribution by lemma 7 and then the uniform to any other distribution by lemma 8. Again from theorem 1 from Hyvärinen and Pajunen (1999) in lemma 8 $F(Y)$ is uniformly distributed so is $G(F(Y)) = Z$, from lemma 7. Since both functions are monotonic, a composition of the two monotonically increasing functions is also a monotonic increasing function and can be mapped via a bijector. According to lemma 7, there exists a sequence of functions, $\{y^z\}_t(y^a)$ that converge to $\sigma \circ G \circ F$ uniformly, thus pointwise. Then there exists a $K_n = \sigma^{-1} \circ \{y^z\}_t(y^a)$ that converges pointwise to $G \circ F$, by monotonicity of $sigma^{-1}$. Then by lemma 2 we have $K_n \to Z$ in distribution. ■

Department of Economics, University of Michigan, Ann Arbor